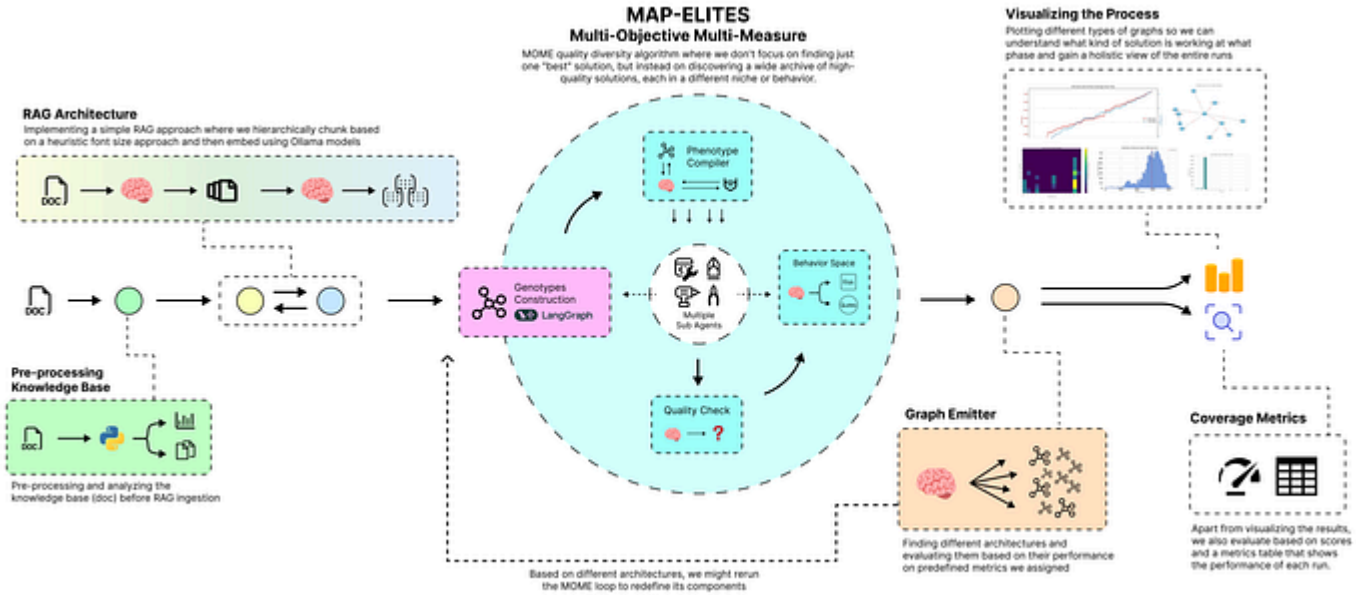


Automating the Search for Optimal LangChain Agent Architectures with Quality-Diversity Algorithm

MAP-Elites, Phenotype Compiler, MOME Archive and more

Fareed Khan

Level Up Coding ~41 min read · September 1, 2025 (Updated: September 1, 2025) ·
Free: No

Read this story for free: link

Developers can optimized architecture for AI agents or RAG systems **but there's a limit to how far this can go by hand.** Exploring thousands of possible designs to find ones that are both diverse and high-performing is practically impossible.

Quality-Diversity (QD) based algorithmic approaches, such as **Enhanced MAP-Elites**, are now appearing as part of AI-based solutions to tackle this challenge and open up new possibilities for modern AI systems ...

In this blog, we will create a complete pipeline for one of the famous Quality-Diversity algorithms, **Enhanced MAP-Elites** using LangChain and pyribs.

You might be new to **Quality-Diversity** term, so we will start by ...

> Understanding the theoretical concepts behind QD algorithms from basics then implement them as a LangChain based RAG and agent system, and finally visualize and analyze their performance.

GitHub - FareedKhan-dev/qd-langchain-agents: Evolving LangChain agent architectures using the...

Evolving LangChain agent architectures using the Quality-Diversity (QD) algorithm. - FareedKhan-dev/qd-langchain-agents

github.com

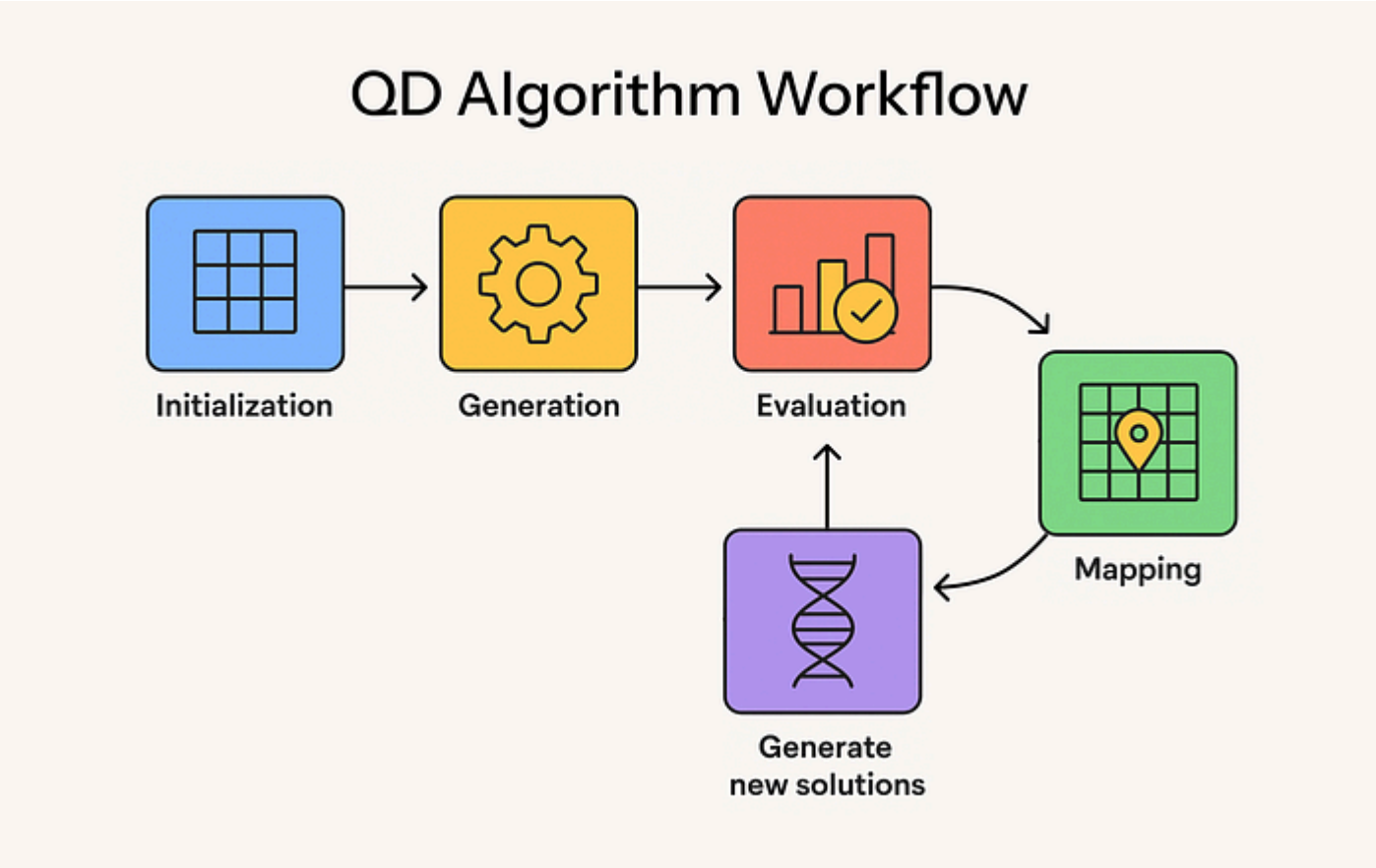
Our codebase is organized as follows:

```
Copy

qd-langchain-agents/
├── LICENSE
├── README.md
├── qd_algorithm_api.ipynb     # For API-based LLMs interactions
├── qd_algorithm_ollama.ipynb  # For local Ollama interactions
└── requirements.txt
```

Table of Contents

What is Quality Diversity Algorithm?

architectures. Let's visualize a generic QD algorithm process first.



QD Workflow (Created by Fareed Khan)

- **We start by initializing the archive:** an empty grid where each cell stands for a specific combination of behaviors like risk level and legal area.

blueprint of an agent, represented as a genotype string that encodes its design.

- **Next we evaluate the solutions**: giving each one a quality score (how strong it is) and a behavior score (what kind of solution it represents).
- **We then map each solution to the archive**: placing it in the cell that matches its behavior. If it outperforms what's already there, it stays, if not, it's discarded.
- **Finally, we generate new solutions from the archive**: selecting top-performing elites, applying small random mutations, and feeding them back into the loop of evaluation, mapping, and improvement until the archive is filled with diverse, high-quality solutions.

There are many QD algorithm you can look into pyribs implementations with examples, **The algorithm we are going to use is an advanced variant of MAP-Elites with a MOME (Multi-Objective Multi-Measure) archive.** Its can handle multiple, often competing, objectives at once.

> Normal QD algorithms make you optimize for one feature, but a MOME archive can keep both like an agent that argues well and one that follows rules.

Pre-processing and Analyzing the TBMP Dataset

The main idea of this pipeline is exploration of different possible solutions. Since exploration is about testing various approaches, we need a complex pipeline.

Using a simple, less complex dataset to showcase this approach would result in overfitting, which leads to a quicker but greedy solution to our problem (fitting only the training data), rather than proper exploration and exploitation.

For that reason, we will use a very complex real-world dataset: the US trademark TBMP 2024. This dataset is kind of a document that decides certain disputes about trademarks (such as when two companies argue over who can use a name or logo).

> The TBMP is the rulebook/manual that explains how these cases work.

It contains a detailed set of rules and discussions, making every page of its content valuable. Let's load this manual from the official US site and save it into our local directory.

```
import requests

# URL for the official TMEP PDF document.
pdf_url = "https://www.uspto.gov/sites/default/files/documents/tbmp-Master-June
# Define the local file path for the downloaded PDF.
pdf_path = "TMEP.pdf"

# Inform the user that the download is starting.
print(f"Downloading TMEP from {pdf_url}...")

# Send an HTTP GET request to the specified URL.
response = requests.get(pdf_url)
# Check if the download was successful and raise an error if not.
response.raise_for_status()

# Open the local file in write-binary mode and save the content.
with open(pdf_path, 'wb') as f:
    f.write(response.content)
```

We need to first check how long our document is by printing its total number of pages.

```
                                    Copy

import fitz  # PyMuPDF

# Open the downloaded PDF file to get its properties.
```

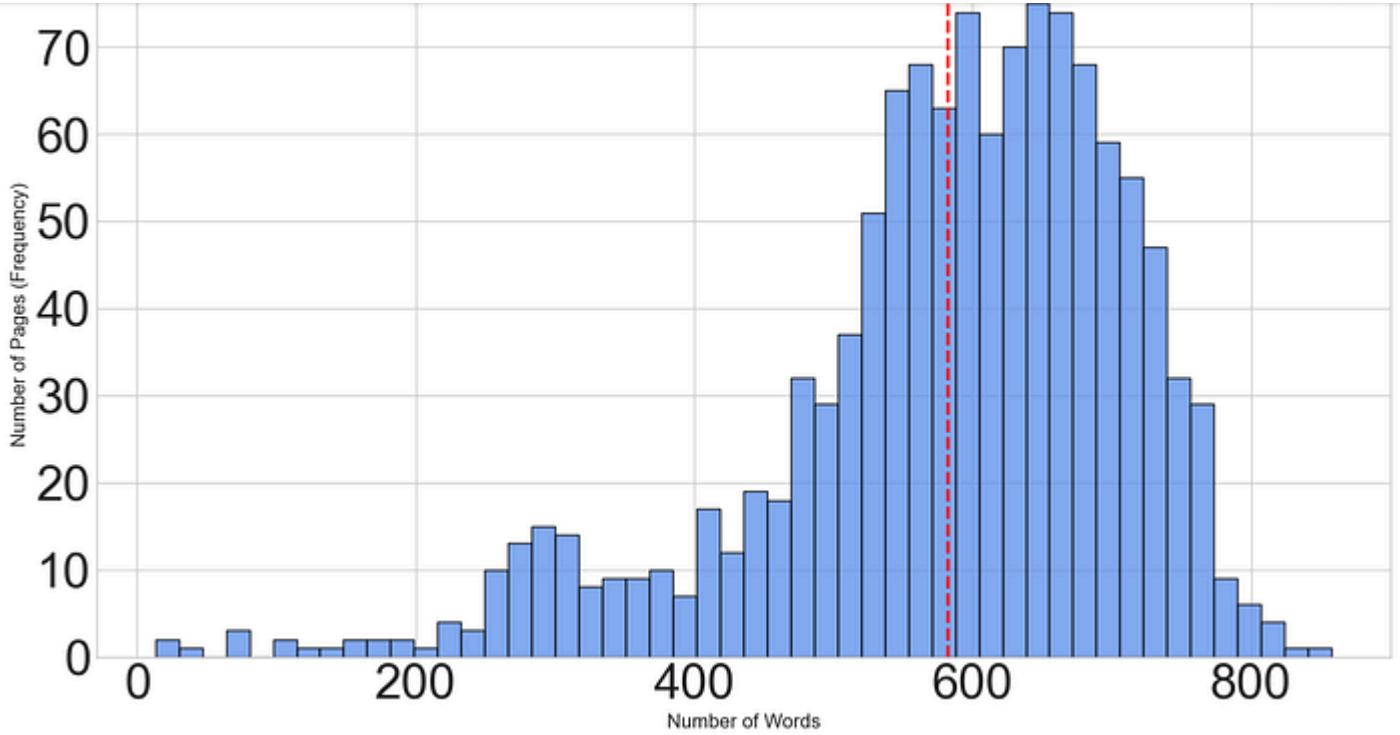
```
num_pages = doc.page_count
# Close the document to free up resources.
doc.close()

# Print the total number of pages.
print(f"The TMEP document has {num_pages} pages.")


#### OUTPUT ####
The TMEP document has 1194 pages.
```

Our document has a little over 1000 pages, which is huge considering that later we will be coding some agents and implementing RAG. Let's plot the distribution of word counts per page in our document.

Copy

```
# plot histogram per page distribution
import matplotlib.pyplot as plt

# Create a list to hold the number of words on each page
words_per_page = []

# Use a more modern plot style
plt.style.use('seaborn-v0_8-whitegrid')

# Create the plot
```

```
# Plot the histogram with more bins and a different color
plt.hist(words_per_page, bins=50, color='cornflowerblue', alpha=0.8, edgecolor=

# Add a vertical line for the mean
mean_words = sum(words_per_page) / len(words_per_page)
plt.axvline(mean_words, color='red', linestyle='dashed', linewidth=2, label=f'M

# Add titles and labels with adjusted font sizes
plt.title('Distribution of Words per Page in TMEP Document', fontsize=16, fontw
plt.xlabel('Number of Words', fontsize=12)
plt.ylabel('Number of Pages (Frequency)', fontsize=12)

# Add a legend
plt.legend()

# Improve layout
plt.tight_layout()

# Display the plot
plt.show()
```



Words distribution (Created by Fareed Khan)

Each page of our document contains roughly 600 words, and since the document has around 1,100 pages, our first task is to convert the dense TMEP PDF into a structured, searchable knowledge base.

If we break the document strictly by pages (an illogical approach), it would result in 1100 chunks, each with about 600 words.

correlated and necessary to answer a query.

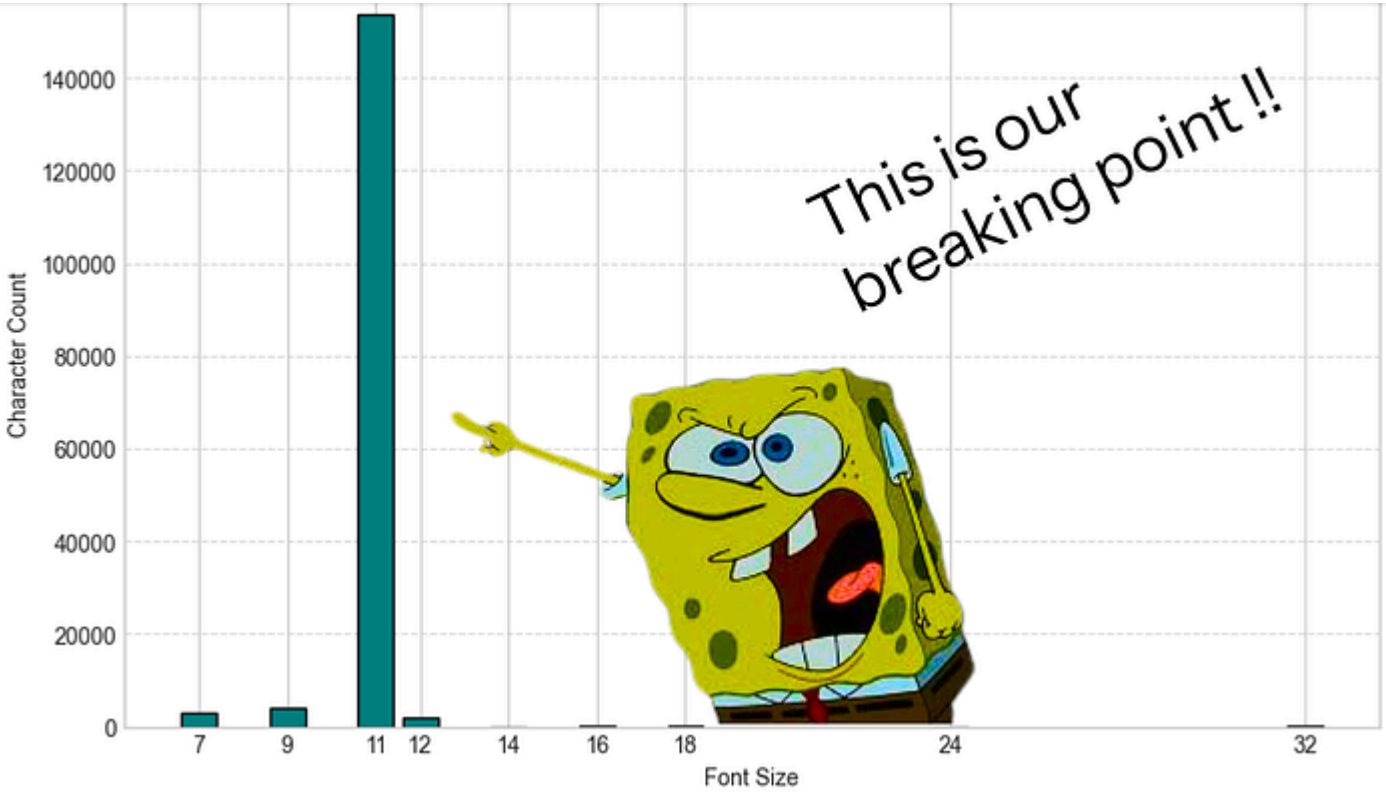
In both unstructured and structured documents, information is usually separated by headings or subheadings. We can use this to split the document more meaningfully.

However, given the length of this document, we need to programmatically classify what qualifies as a heading and what does not. The best way to approach this is to visually analyze the structure of the document first.

Copy

```
# Import Counter for tallying font sizes.
from collections import Counter

# Initialize a Counter to store font sizes and their corresponding character co
font_counts = Counter()

# To speed up the analysis, we'll only process the first 50 pages.
# This provides a representative sample of the document's font usage.
analysis_page_limit = 50

# Iterate through each page of the document up to the specified limit.
for page_num, page in enumerate(doc):
    if page_num >= analysis_page_limit:
```

```
    # Extract detailed text information, including font size, as a dictionary.
    blocks = page.get_text("dict")["blocks"]
    for b in blocks:
        if b['type'] == 0:  # A text block.
            for l in b["lines"]:
                for s in l["spans"]:
                    # Round the font size to handle minor variations.
                    font_size = round(s['size'])
                    # Add the number of characters to the count for this font s
                    font_counts[font_size] += len(s['text'])

# Get the 10 most common font sizes and their counts.
top_10_fonts = font_counts.most_common(10)
sizes = [item[0] for item in top_10_fonts]
counts = [item[1] for item in top_10_fonts]

# Create a bar chart to visualize the font distribution.
plt.figure(figsize=(10, 6))
plt.bar(sizes, counts, color='teal', edgecolor='black')
plt.xlabel('Font Size')
plt.ylabel('Total Character Count')
plt.title(f'Top {len(sizes)} Most Common Font Sizes (First {analysis_page_limit
plt.xticks(sizes)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Font Size Check (Created by Fareed Khan)

The plot clearly shows that the normal font size of the document is 11, and any font size above that can reasonably be considered a heading. We can use this criterion to divide our document into sections and build a structured knowledge base.

```
Copy
```

```
# Titles are typically larger than body text. We'll define a title as any text
# with a font size at least 15% larger than the body font.
title_threshold = body_font_size * 1.15

# Print the determined values for verification.
print(f"\nDetermined Dominant Body Font Size: {body_font_size}")
print(f"Setting Title Font Size Threshold to > {title_threshold:.2f}")

#### OUTPUT ####
Determined Dominant Body Font Size: 11
Setting Title Font Size Threshold to > 12.65
```

We are setting the threshold so that anything above font size 11 will be considered a heading. Now, we need to parse the document according to this logic.

Copy

```
from collections import namedtuple

# A simple data structure to hold our parsed elements.
PDFElement = namedtuple("PDFElement", ["type", "text"])
```

apply the chunking logic accordingly.

Copy

```
# Loop through each page in the document with its index
for page_num, page in enumerate(doc):
    # Extract text as a dictionary structure and get the list of blocks on the
    blocks = page.get_text("dict")["blocks"]

    # Iterate over each block on the page
    for b in blocks:
        if b['type'] == 0:  # Only process text blocks (ignore images, drawings
            block_text = ""     # Accumulator for all text in the block
            span_sizes = []     # Store font sizes of each text span

            # Traverse lines and spans (small text runs with same style) within
            for l in b["lines"]:
                for s in l["spans"]:
                    block_text += s['text'] + ' '  # Concatenate span text with
                    span_sizes.append(s['size'])  # Record font size of the spa

            # Classify the block:
            # If ALL spans are larger than the title threshold → mark as "title
            # Otherwise → mark as "text"
            element_type = "title" if all(size > title_threshold for size in sp
```

Let's now print a sample to verify that the parsing has been done correctly.

```
Copy

# Get the first element classified as a "title" (if any exist).
title_sample = next((el for el in elements if el.type == 'title'), None)

# Get the first "text" element with more than 200 characters (to ensure it's a
text_sample = next((el for el in elements if el.type == 'text' and len(el.text)

# Show type and the first 100 characters of the title text for inspection
print(f"SAMPLE TITLE: Type: {title_sample.type}, Text: '{title_sample.text[:100

# Show type and the first 200 characters of the text for inspection
print(f"SAMPLE TEXT: Type: {text_sample.type}, Text: '{text_sample.text[:200]}.
```

This is the output we get.

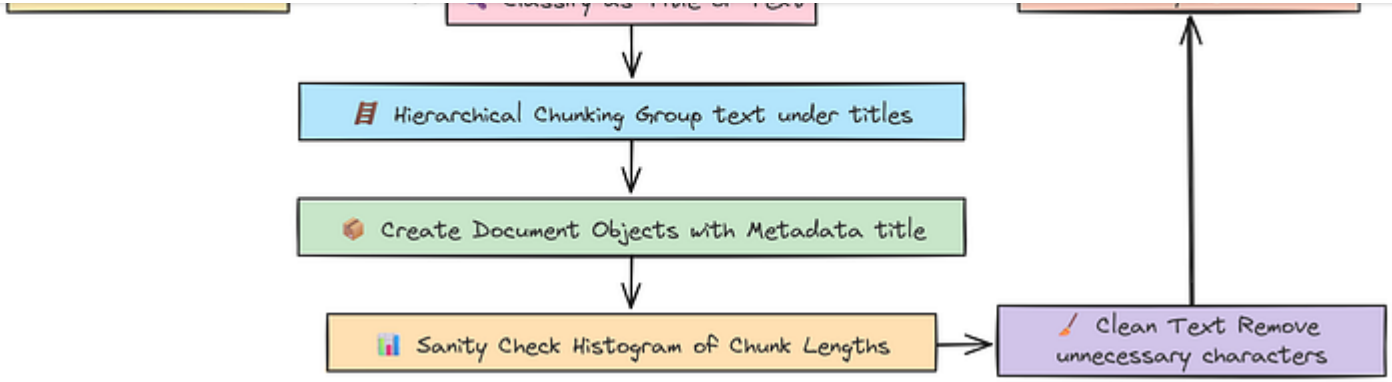
```
Copy

# --- Sample Parsed Elements (showing both types) ---
SAMPLE TITLE: Type: title, Text: TRADEMARK TRIAL AND APPEAL BOARD MANUAL OF PRO
SAMPLE TEXT: Type: text, Text: The June 2024 revision of the Trademark Trial ar
```

dataset.

Hierarchical Chunking Using Font-Based Heuristics

Okay, great. Now that we have our text elements neatly classified as either a `title` or `text`, we can do something much smarter than just splitting the document by page or character count. We are going to group our text contextually, using the document's own structure as our guide.

The logic here is simple but very effective. We will iterate through our list of elements. All the `text` parts will be collected together under whatever the most recent `title` was. We then store that title in the document's metadata. This is really important because it means when our RAG system later pulls out a piece of information, it will also know the section it came from, giving our AI some crucial context to work with.



HC Explanation (Created by Fareed Khan)

Here, we are basically creating a function that will build these structured `Document` objects for us.

Copy

```
from langchain_core.documents import Document

def create_hierarchical_chunks_from_custom_elements(elements):
    chunks = []
    current_chunk_text = ""
    current_metadata = {}

    for element in elements:
        # When we find a title, it signals the start of a new section.
        if element.type == 'title':
            # If we already have text from the *previous* section, save it as a
            if current_chunk_text:
```

```
            # Now, start the new chunk. The title becomes the metadata and the
            current_metadata["source_title"] = element.text
            current_chunk_text = element.text + "\n\n"
        # If the element is just text, we append it to the current section.
        elif element.type == 'text':
            current_chunk_text += element.text + "\n\n"

    # After the loop, we need to save the very last chunk that was being built.
    if current_chunk_text:
        chunks.append(Document(page_content=current_chunk_text.strip(), metadat

    return chunks

# Let's create our final list of Document objects using the function.
tmep_chunks = create_hierarchical_chunks_from_custom_elements(parsed_elements)

print(f"Created {len(tmep_chunks)} hierarchical chunks.")


#### OUTPUT ####
Created 87 hierarchical chunks.
```

Instead of 1194 chunks (one per page), our structure-aware method has created 87 large, contextually rich chunks. Each one of these represents a major section from the original manual. Let's print out a full sample to be absolutely sure our logic worked as intended.

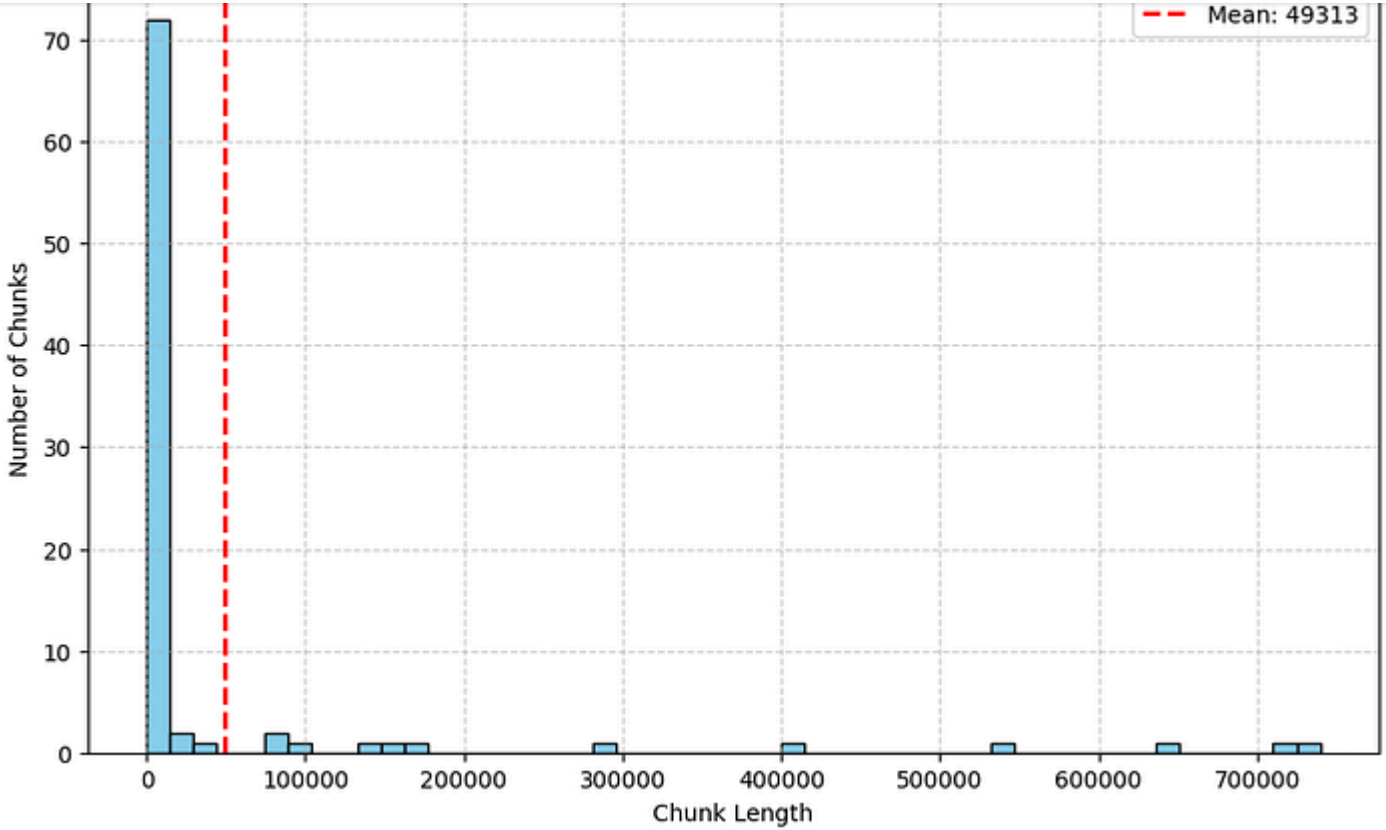
```
# --- Sample chunk (full content) ---
sample_chunk_index = 44
sample_chunk = tmep_chunks[sample_chunk_index]
print(f"Metadata for chunk {sample_chunk_index}: {sample_chunk.metadata}")
print("Content:")
print(sample_chunk.page_content)


#### OUTPUT ####
Metadata for chunk 44: {'source_title': 'INDEX'}
Content:
INDEX

Discovery Depositions: — Electronic Signature

Sec. No. Sec. No.

Electronically Stored Information : Discovery of.....
...
```

The output confirms it perfectly. The metadata correctly identifies the chunk's title as **'INDEX'**, and the content is the full text from that section. This is exactly what we wanted.

Before we move on to embedding, It's better to do a quick sanity check. Let's just plot a histogram of our final chunk lengths. An ideal distribution avoids having

Copy

```
# For mathematical Calculation
import numpy as np

# We'll get the length of each document's page_content in characters.
chunk_lengths = [len(chunk.page_content) for chunk in tmep_chunks]

# Now, let's create a histogram to see the distribution.
plt.figure(figsize=(10, 6))
plt.hist(chunk_lengths, bins=50, color='skyblue', edgecolor='black')
plt.title('Distribution of TMEP Chunk Lengths (PyMuPDF) (in characters)')
plt.xlabel('Chunk Length')
plt.ylabel('Number of Chunks')

# Adding a line for the mean gives us a good reference point.
mean_len = np.mean(chunk_lengths)
plt.axvline(mean_len, color='red', linestyle='dashed', linewidth=2, label=f'Mea
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Chunks length (Created by Fareed Khan)

The distribution shows that most of our chunks are of substantial size, which can be a problem when using models with a small context window. However, recent LLMs now support larger context windows, so this issue can be addressed.

Before moving on to embeddings, we need to address a small issue: our chunks contain unnecessary characters like `"."`, which only increase the context size without adding any meaning. So, let's clean our text a bit.

Copy

```
# Remove all periods from the page_content of each chunk.
for chunk in tmep_chunks:
    chunk.page_content = chunk.page_content.replace('.', '')

# Display the modified content of a sample chunk to see the result.
print(tmep_chunks[44].page_content)
```

This is what our cleaned text info looks like.

Copy

```
# Sample Cleaned output

INDEX
Discovery Depositions: – Electronic Signature=
Sec No Sec No
Electronically  Stored  Information : Discovery of § 40202 Duty to S ...
```

Indexing the Knowledge Base

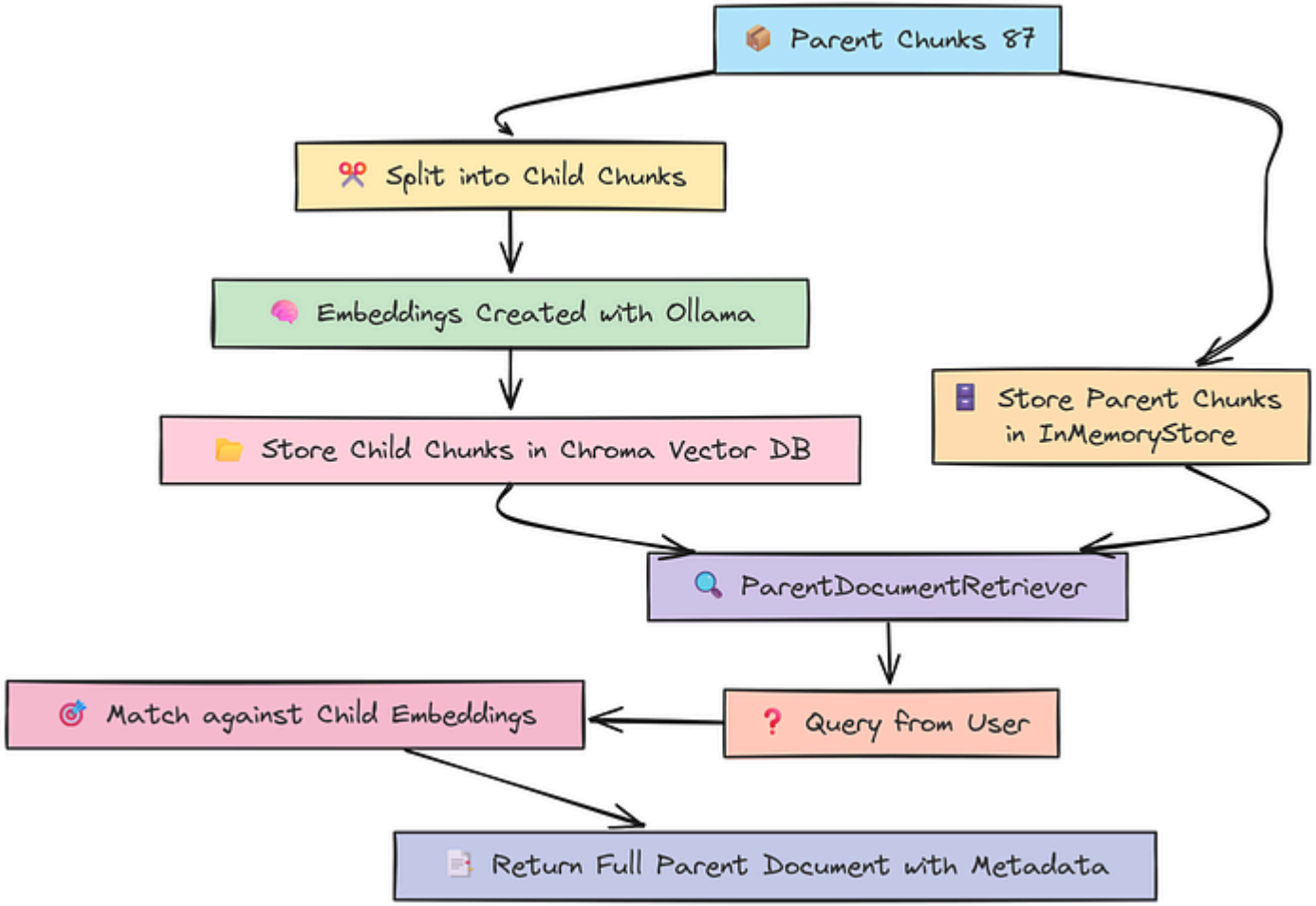
Alright, now that our knowledge base is intelligently chunked, it's time to build the actual retrieval system. This is the final and most crucial step in preparing the foundation for our evolutionary agents.

We can't just throw these chunks into a standard vector store, for a problem this complex, we need a more sophisticated strategy.

This is why we will be using the `ParentDocumentRetriever`. It's a RAG component that balances precision with context.

Here is the main idea and why it's so important for our QD pipeline:

1. **Splitting:** Each of our large, hierarchical chunks (the **"parent"** documents) will be split into even smaller "child" chunks.
2. **Embedding:** We will only create vector embeddings for these small, focused child chunks using ollama embedding models.
3. **Storing & Retrieving:** When a query comes in, it's compared against the embeddings of the small, precise child chunks. This leads to very accurate



Indexing Flow (Created by Fareed Khan)

This gives our agents two ways to support their generated answer ...

Let's set this up. We will initialize our `OllamaEmbeddings`, a `Chroma` vector store for the child chunks, and an in-memory store for our large parent documents.

Copy

```
from langchain_ollama import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain.text_splitter import RecursiveCharacterTextSplitter
from tqdm.auto import tqdm

# Initialize the embedding model from OLLAMA AI.
embeddings = OllamaEmbeddings(model="BAAI/bge-en-icl")

# This is the vector store for the small, embedded child chunks.
vectorstore = Chroma(collection_name="tmep_rules_ollama", embedding_function=em
# This is the simple in-memory store for our large, original parent chunks.
docstore = InMemoryStore()
# This splitter will create the small child chunks from our parents.
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)

# The main retriever object that orchestrates the entire process.
retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=docstore,
```

Now that the components are ready, we can add our `tmep_chunks`. The retriever will automatically handle the process of splitting them into children, embedding the children, and storing everything in the right place. We are going to index all 87 chunks.

```
Copy

# indexing all tmep_chunks.
retriever.add_documents(tqdm(tmep_chunks), ids=None)


#### OUTPUT ####
Setting up the vector store with ParentDocumentRetriever and OllamaEmbeddings..

Adding documents to the retriever. This will handle chunking, embedding, and ir
100%|██████████| 87/87 [00:00<00:00, 4.88it/s]
```

Let's run a quick test to see it in action. I'll ask a specific legal question and check what it retrieves.

```
Copy
```

```
retrieved_docs = retriever.get_relevant_documents(test_query)

print(f"Retrieved {len(retrieved_docs)} documents for the query: '{test_query}
if retrieved_docs:
    print(f"\nTop result (parent document) metadata: {retrieved_docs[0].metadat
    print(f"Top result content (first 400 chars):\n{retrieved_docs[0].page_cont
```

This is what the retrieved info looks like.

```
Copy

# --- Testing Retriever ---
Retrieved 2 documents for the query: 'What are the DuPont factors for likelihoc

Top result (parent document) metadata: {'source_title': 'TRADEMARK TRIAL AND AP
Top result content (first 400 chars):
TRADEMARK TRIAL AND APPEAL BOARD MANUAL OF PROCEDURE (TBMP)
June 2024
US Department of Commerce
United States Patent and Trademark Office
Foreword
The June 2024 revision of the Trademark Trial and Appeal Board Manual of Proced
```

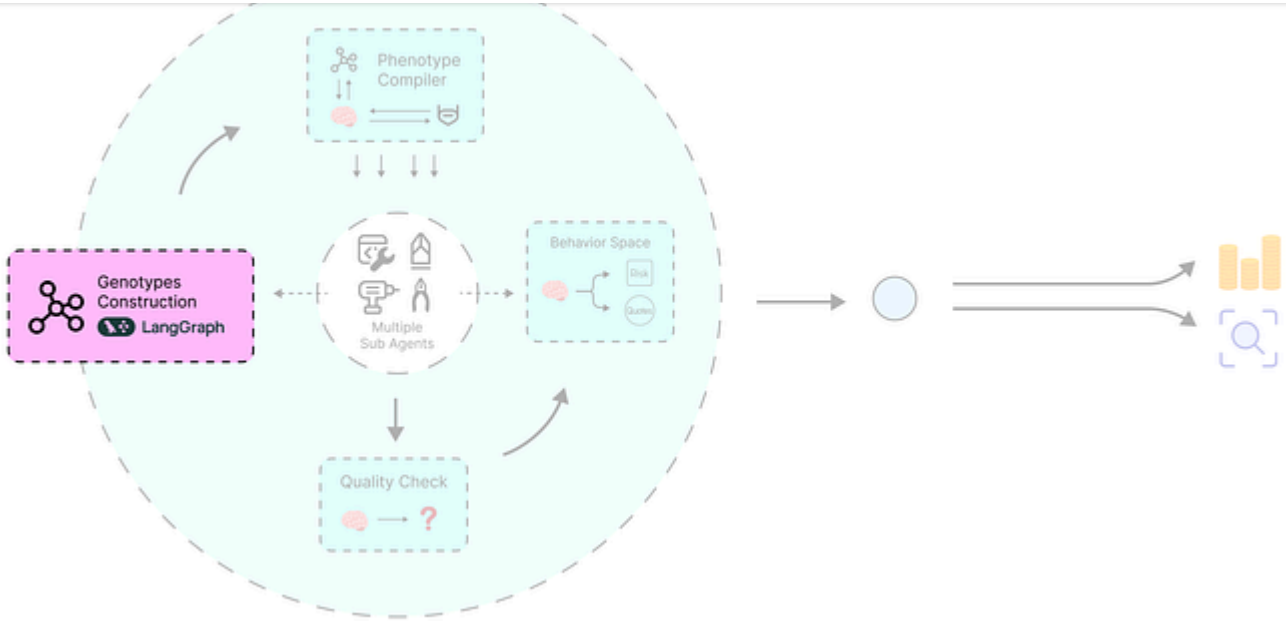
The retriever found relevant information and, as expected, it returned the full parent document, complete with its metadata. Our knowledge base is now ready.

Defining the LangGraph Agent Genotypes

With our knowledge base ready, we can start defining the core of our evolutionary system.

> The whole point of the Quality-Diversity algorithm is to search a space of possible solutions.

For us, a "solution" isn't just a final answer, it's the **architectural blueprint of the AI agent itself.**



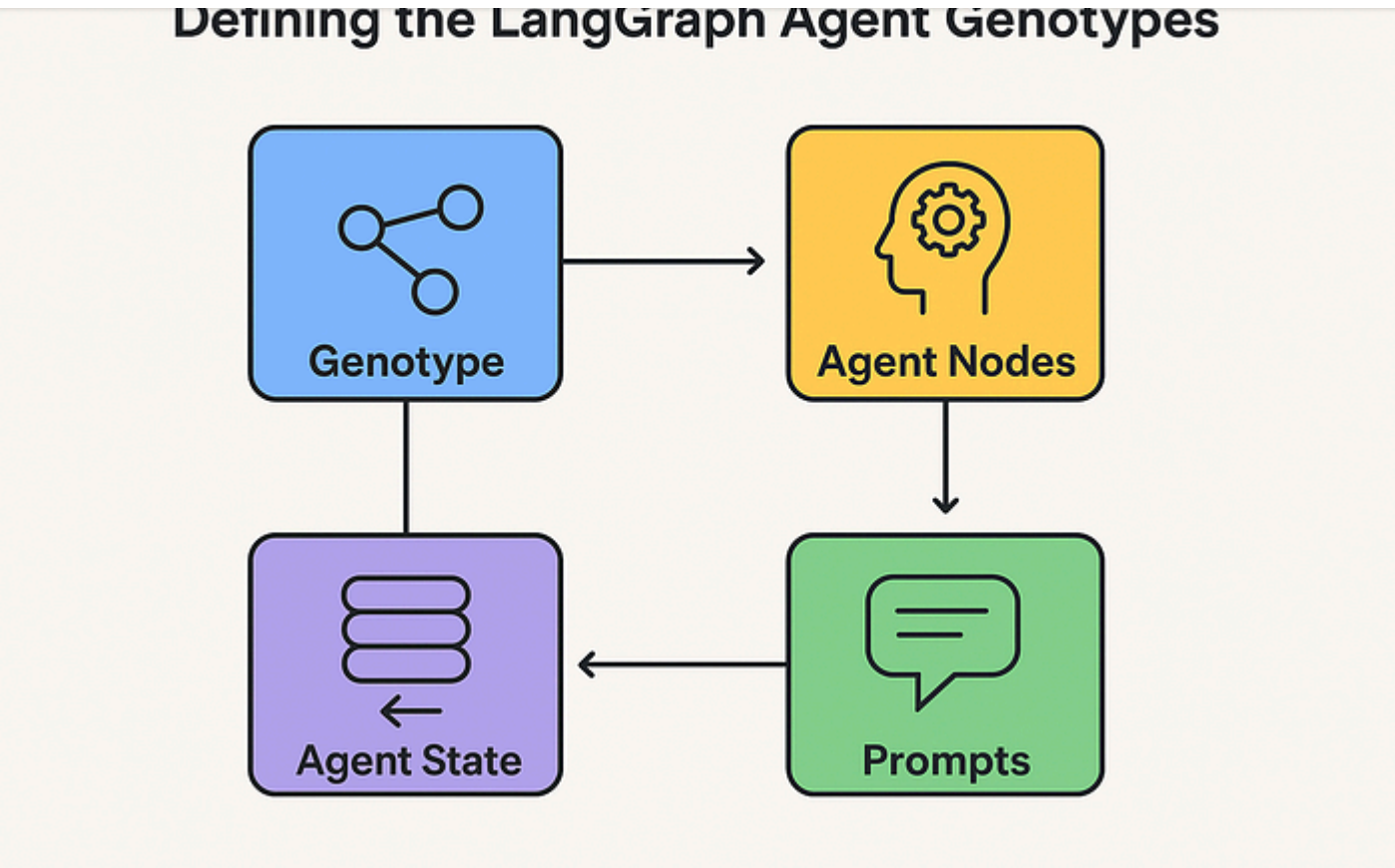
Genotypes construction (Created by Fareed Khan)

This blueprint is what we call the **genotype**. In our system, we're going to represent this genotype as a simple string that defines a directed graph. Each node in this graph will be a specialized AI agent (like a `Case_Analyzer` or a `Risk_Assessor`), and the edges will define how information flows between them. Let's initialize our ollama chat llm.

```
Copy
```

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated, List
import operator

# Initialize the LLM from Ollama AI that will power all our agent nodes.
llm = ChatOllama(model="deepseek-ai/DeepSeek-V3", temperature=0.3)
```

The `pyribs` QD algorithm will work by creating and modifying these genotype strings, searching for the agent architectures that produce the most diverse and high-quality legal arguments.



Genotype Flow (Created by Fareed Khan)

First things first, let's define the fundamental components of our agents. This involves two key pieces:

```
Copy
```

```
    "Case_Analyzer": ChatPromptTemplate.from_template(
        "You are a senior trademark attorney. Your task is to analyze the follo
        "Focus on identifying the core legal issue and suggest a specific angle
        "Office Action: {office_action}\nPrior Research Notes:\n{research_notes
    ),
    "Legal_Researcher": ChatPromptTemplate.from_template(
        "You are a paralegal. Based on the proposed strategy, retrieve relevant
        "Your output should be the key information that supports this strategy.
    ),
    "Argument_Crafter": ChatPromptTemplate.from_template(
        "You are a junior attorney. Draft a persuasive legal argument respondir
        "The argument should be clear, well-structured, and directly address th
        "Office Action: {office_action}\nResearch Notes:\n{research_notes}"
    ),
    "Risk_Assessor": ChatPromptTemplate.from_template(
        "You are an opposing counsel. Your job is to critique the following leg
        "Be critical and objective.\n\nArgument:\n{argument}"
    ),
}
```

1. **The Agent Nodes & Prompts:** We need to define the "persona" and task for each possible role our agent can have. We'll set up our `ChatOllama` model and create a dictionary of `ChatPromptTemplate` objects, one for each role like `Case_Analyzer`, `Legal_Researcher`, and so on.
2. **The Agent State:** This is the shared memory or "scratchpad" that all the nodes in our agent graph will use. We'll define it using a `TypedDict`. As the agent

which the `Argument_Crafter` can then read to build its case.

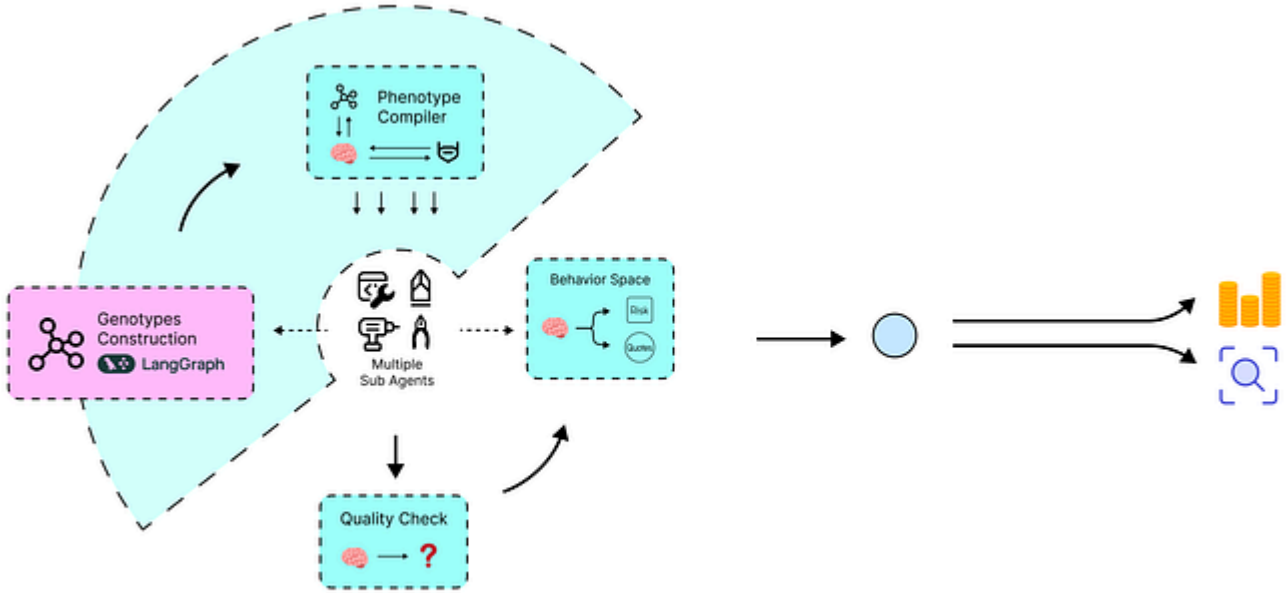
We can now combines these component into one single legal agent state OOP.

```
Copy

# Now, we define the structure of the agent's shared memory (state).
class LegalAgentState(TypedDict):
    office_action: str  # The initial problem statement
    strategy: str       # High-level plan from the analyzer
    # Research notes can accumulate from multiple steps, so we define it to app
    research_notes: Annotated[List[str], operator.add]
    argument: str       # The final drafted argument
    critique: str       # The critique from the risk assessor
```

With these fundamental building blocks, our potential agent roles and their shared memory structure, we can now build the compiler that will turn our genotype strings into actual, runnable LangGraph agents.

Genotype-to-Phenotype Compiler

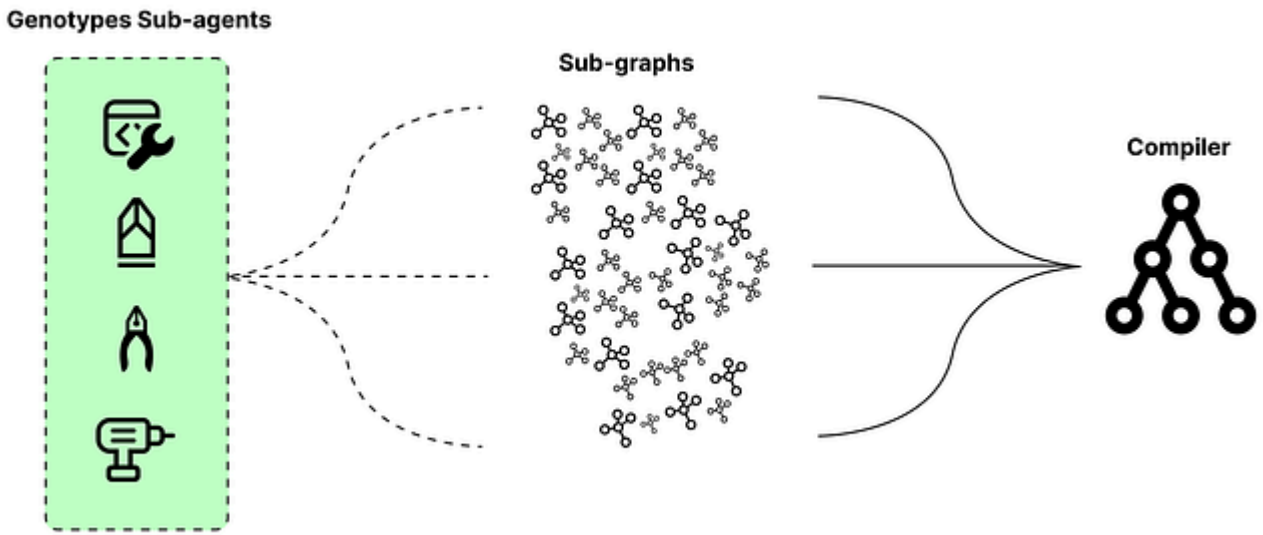
This next part we want to code is the real component of our system. We have the agent's DNA (the genotype string), but we need a way to turn that blueprint into a



Genotype-to-Phenotype Compiler (Created by Fareed Khan)

We need to have a function `compile_genotype_to_graph` that will take a genotype string as input and dynamically construct a runnable `LangGraph` agent from it.

This is a very important concept ...



Phenotype compiler (Created by Fareed Khan)

Let's start building it. The first step inside our compiler is to define the functions that each possible node in our graph can execute. Each function simply takes the current state, runs its assigned LLM chain, and returns the result to update the state. We'll also create a `node_map` to easily look up these functions by their string name.

Copy

```
    workflow = StateGraph(LegalAgentState)

    # Define the functions that will be executed by each node.
    def run_case_analyzer(state): return {"strategy": (prompts["Case_Analyzer"]
    def run_legal_researcher(state): return {"research_notes": ["\n".join([d.pa
    def run_argument_crafter(state): return {"argument": (prompts["Argument_Cra
    def run_risk_assessor(state): return {"critique": (prompts["Risk_Assessor"]

    # We map the string names from our genotype to their corresponding functior
    node_map = {
        "Case_Analyzer": run_case_analyzer,
        "Legal_Researcher": run_legal_researcher,
        "Argument_Crafter": run_argument_crafter,
        "Risk_Assessor": run_risk_assessor
    }
```

Okay, with the node behaviors defined, the next step is to parse the incoming `genotype` string. We'll split the string by semicolons to get the individual edges (like `"Case_Analyzer -> Legal_Researcher"`).

From these edges, we can figure out all the unique nodes we need to create. Once we have the set of required nodes, we add them to our `workflow` graph.

Copy

```
edges = genotype.strip().split(';')

# First, we identify all unique nodes mentioned in the genotype string.
for edge in filter(None, edges):
    parts = edge.split('->')
    if len(parts) != 2: continue
    source, dest = parts
    if source in node_map: nodes.add(source)
    if dest in node_map: nodes.add(dest)

# We add all the identified nodes to our workflow graph.
for node_name in nodes:
    workflow.add_node(node_name, node_map[node_name])
```

Now that the nodes exist in our graph, we just need to connect them. We'll loop through our `edges` again. This time, we'll use `workflow.add_edge` to create the connections. A special case is when a node points to `"END"`, which tells LangGraph that this is a terminal point in the flow.

Finally, we set the entry point for the graph and call `.compile()` to return the finished, runnable agent.

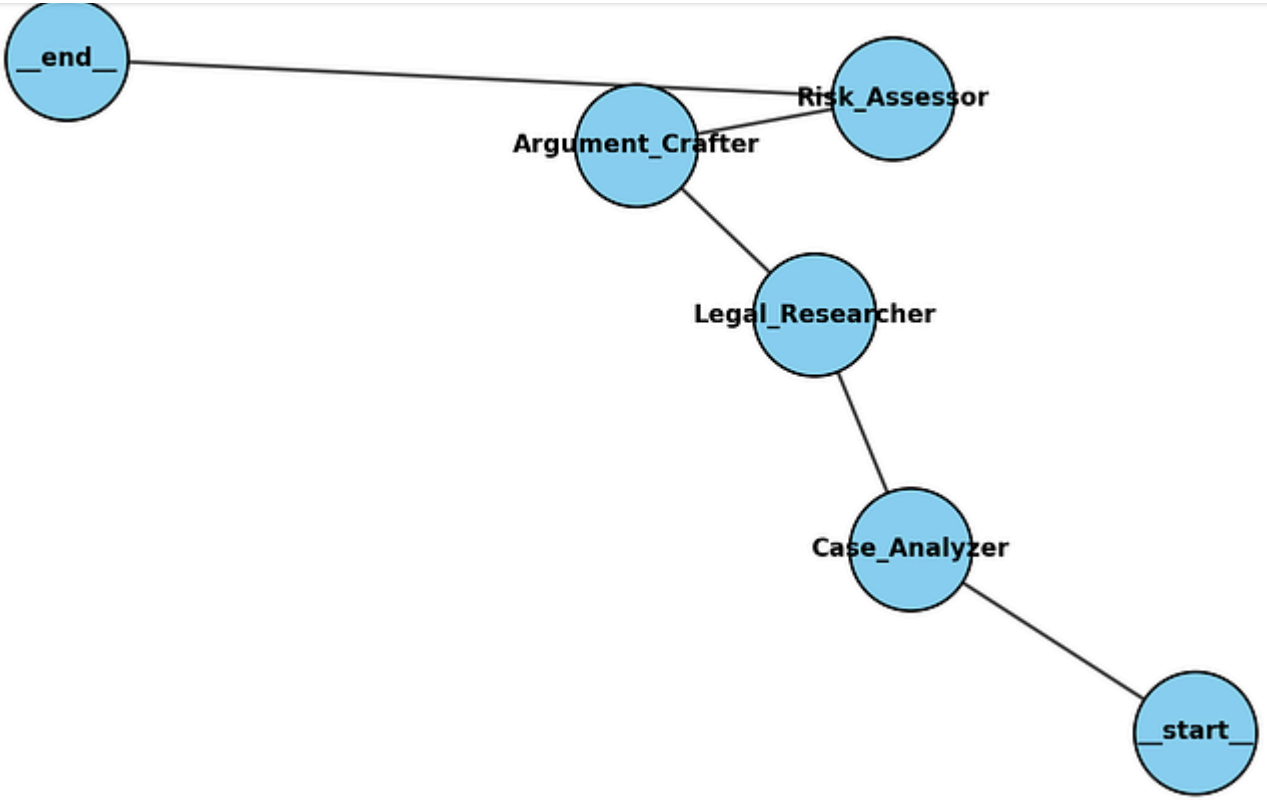
Copy

```
    parts = edge.split('->')
    if len(parts) != 2: continue
    source, dest = parts
    if source in nodes:
        # An edge pointing to "END" connects to the graph's terminal state.
        if dest == "END":
            workflow.add_edge(source, END)
        elif dest in nodes:
            workflow.add_edge(source, dest)

# We set the entry point, which is always the Case_Analyzer.
if 'Case_Analyzer' in nodes:
    workflow.set_entry_point("Case_Analyzer")
else: # A fallback just in case.
    raise ValueError("Genotype must contain a Case_Analyzer to serve as an entr

# Compile the graph into our final, executable agent.
return workflow.compile()
```

With our complete compiler function defined, let's give it that quick test. I'll create a sample genotype for a simple, linear agent. Then we'll use our new compiler to build it and use LangGraph's built-in visualizer to confirm the structure is correct.

```
Copy
```

```
    "Case_Analyzer->Legal_Researcher;"
    "Legal_Researcher->Argument_Crafter;"
    "Argument_Crafter->Risk_Assessor;"
    "Risk_Assessor->END"
)
print(f"Sample Genotype:\n{sample_genotype}\n")

# Use our compiler to turn the string into an agent.
test_agent = compile_genotype_to_graph(sample_genotype)
print(f"Successfully compiled genotype into a runnable LangGraph agent.")

# Print an ASCII art representation of the compiled graph to verify its structu
print("\n--- ASCII Visualization of the Agent Graph ---")
test_agent.get_graph().print_ascii()
```



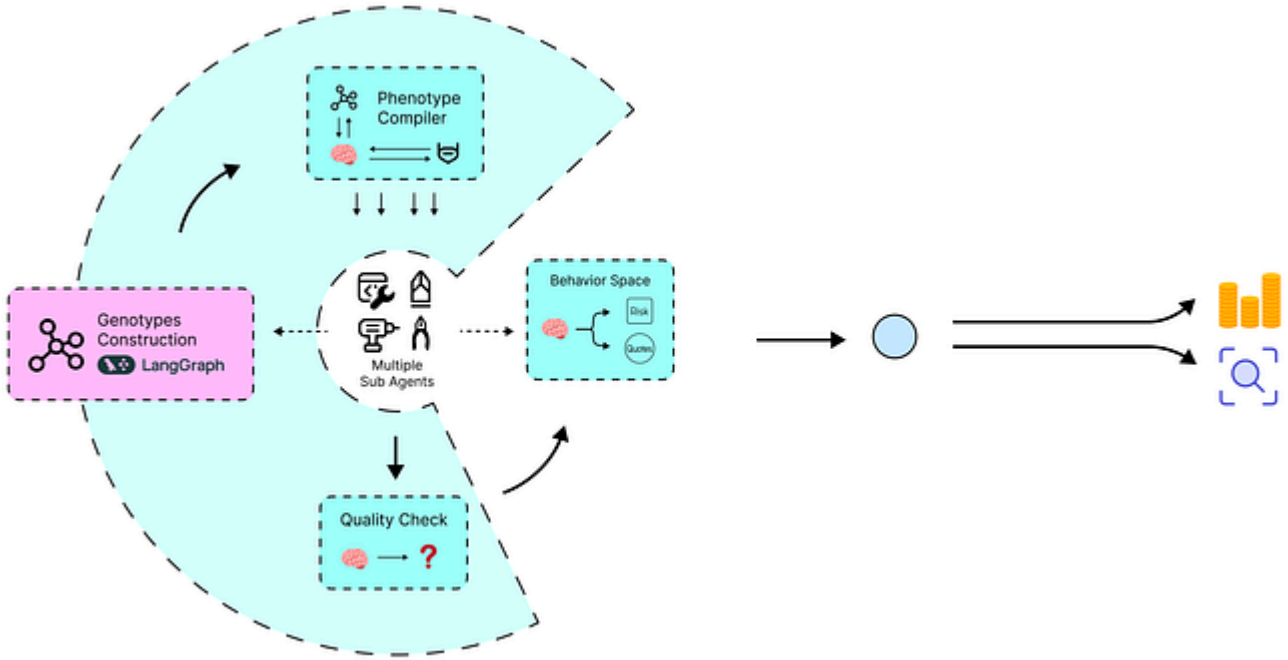
Agentic Workflow (Created by Fareed Khan)

The visualization shows the exact linear flow we defined in our genotype string. Now that we have a reliable way to turn any architectural blueprint into a functional agent, we need a way to *evaluate* the performance of these agents. This brings us to the next component, our AI Moot Court.

Implementing the Quality Function

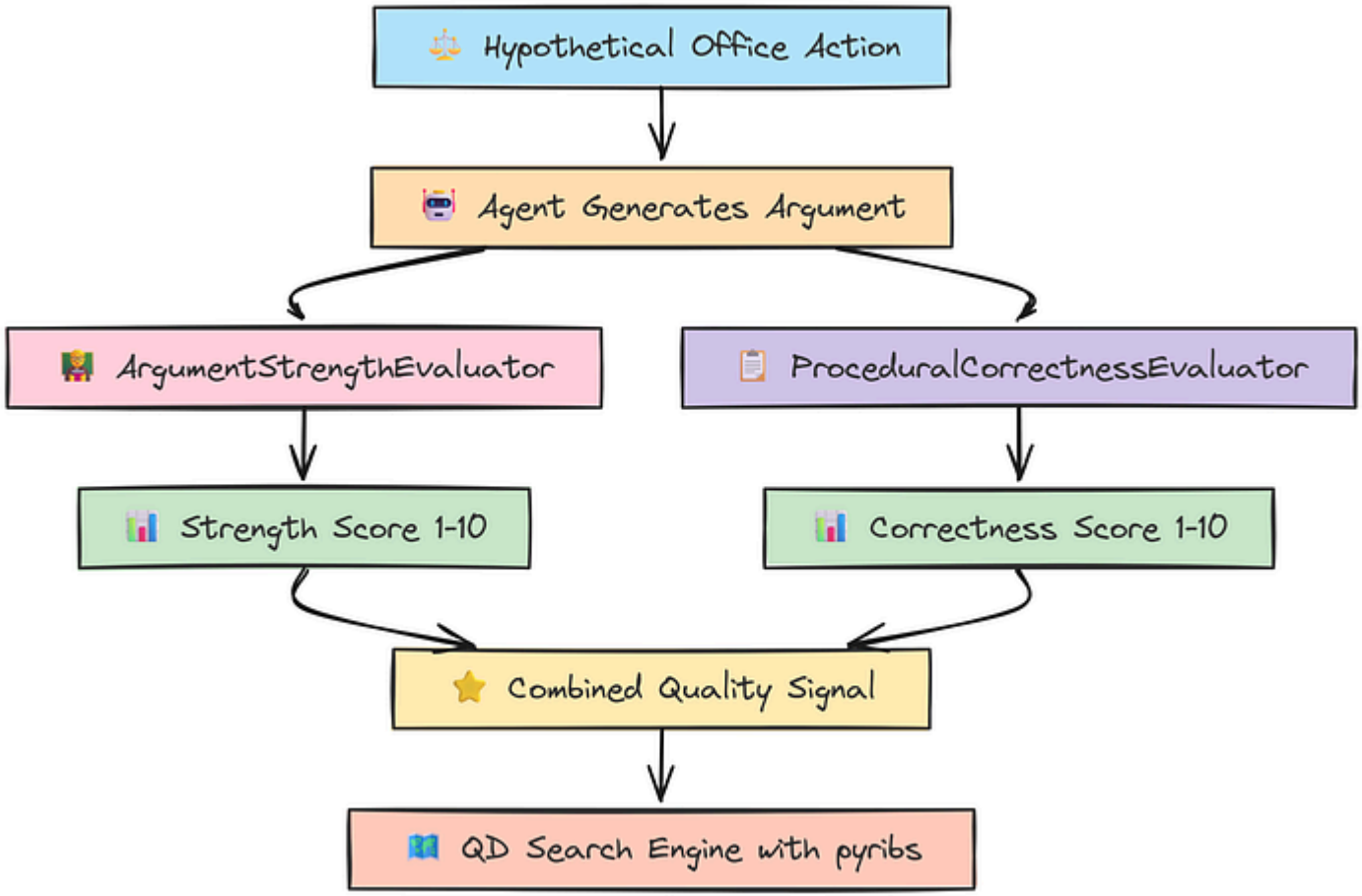
So, we have a way to dynamically build agents with different architectures.

> But how do we know which architectures are "good"? And how do we determine their characteristics (like riskiness) to map them in our QD archive?



Quality Evaluator (Created by Fareed Khan)

This is where `LangSmith` comes in. We're going to set up an "AI Moot Court" a suite of AI-powered judges that will automatically assess the legal arguments generated by our agents. This feedback is the signal that will guide our entire search.



First, we need to define the exact legal problem that every single agent will have to solve. It's really important that this scenario is fixed. By using a constant `HYPOTHETICAL_OFFICE_ACTION`, we ensure that we're fairly comparing the performance of different agent architectures, not just how they handle different problems.

```
Copy

# This is the fixed legal problem every agent will try to solve.
HYPOTHETICAL_OFFICE_ACTION = (
    "Your application for the trademark 'PINNACLE' for 'financial consulting se
    "under Trademark Act Section 2(d) because of a likelihood of confusion with
    "'PINNACLE PARTNERS' for 'investment advisory services', U.S. Reg. No. 1,23
)
```

Next, let's set up the basic tools for our evaluation suite. We'll initialize the `LangSmith` client and a separate, fast `ChatOllama` model specifically for our evaluation tasks.

It's a good practice to use a different model for evaluation than for generation. This helps reduce bias, as the evaluator model is less likely to favor the style of text its own variant produces.

```
from langsmith.evaluation import RunEvaluator
from langsmith.schemas import Example, Run
from langsmith import Client

# Initialize the LangSmith client for programmatic interaction.
client = Client()

# We'll use a fast and cost-effective model for our evaluation tasks.
eval_llm = ChatOllama(model="Qwen/Qwen3-30B-A3B", temperature=0)
```

Now we can create our judges. For the Quality-Diversity algorithm, we need two types of feedback: scores for **Quality** (how good is the solution?) and descriptors for **Behavior** (what kind of solution is it?).

Let's start with the quality evaluators. Their combined scores will tell us how good an agent's argument is. We'll create two:

Copy

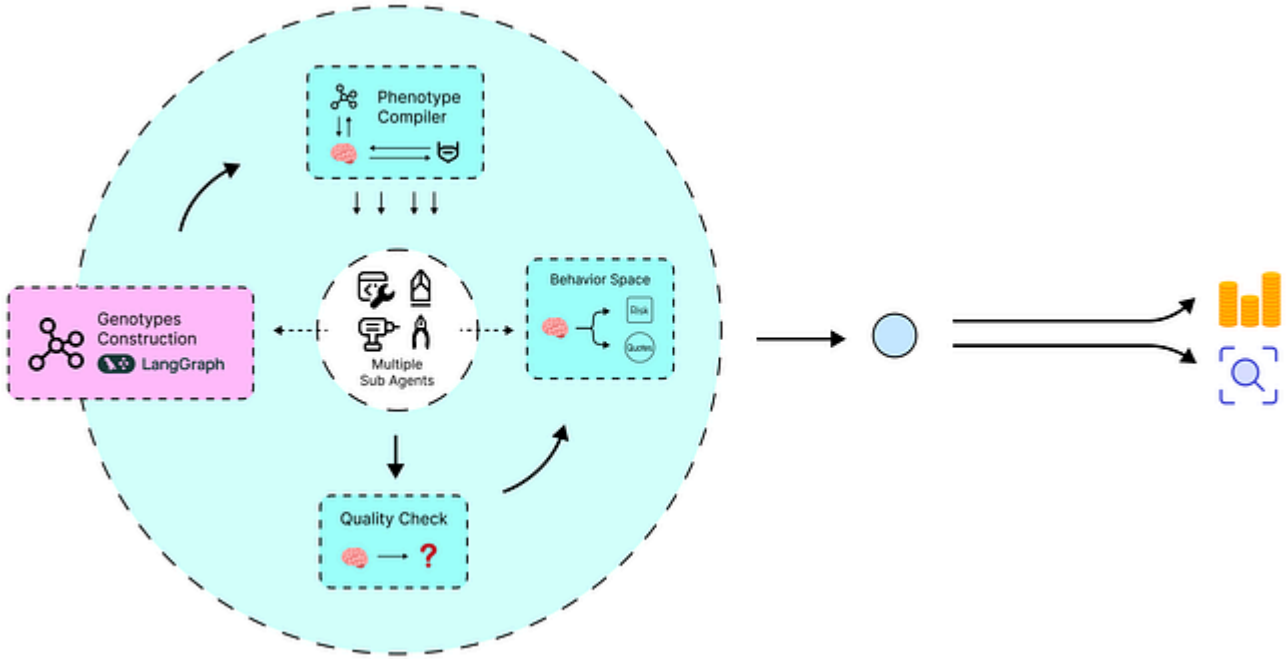
```
# This evaluator scores the legal strength and persuasiveness of the argument.
class ArgumentStrengthEvaluator(RunEvaluator):
    def __init__(self):
        # The chain that will perform the evaluation.
        self.chain = ChatPromptTemplate.from_template(
            "You are a trademark law professor. Evaluate the following legal ar
            "Score it from 1 (very weak) to 10 (very strong). Return ONLY the
```

```
    def evaluate_run(self, run: Run, example: Example | None = None) -> dict:
        argument = run.outputs.get("argument", "") if run.outputs else ""
        if not argument: return {"key": "strength", "score": 0}
        try: score = int(self.chain.invoke({"argument": argument}).content.stri
        except (ValueError, TypeError): score = 0
        return {"key": "strength", "score": score}
# This evaluator scores the procedural and formal correctness of the argument.
class ProceduralCorrectnessEvaluator(RunEvaluator):
    def __init__(self):
        self.chain = ChatPromptTemplate.from_template(
            "You are a meticulous paralegal. Review the following argument for
            "Score it from 1 (procedurally flawed) to 10 (procedurally perfect)
        ) | eval_llm

    def evaluate_run(self, run: Run, example: Example | None = None) -> dict:
        argument = run.outputs.get("argument", "") if run.outputs else ""
        if not argument: return {"key": "correctness", "score": 0}
        try: score = int(self.chain.invoke({"argument": argument}).content.stri
        except (ValueError, TypeError): score = 0
        return {"key": "correctness", "score": score}
```

1. `ArgumentStrengthEvaluator` : This evaluator will act like a law professor, scoring the persuasive power of the argument on a scale of 1-10.
2. `ProceduralCorrectnessEvaluator` : This one will act like a meticulous paralegal, scoring the argument's adherence to legal formalities, also from 1-10.

Defining the Behavior Space

Before we create other types of evaluator we need to accurately classify a legal argument by providing context for each section code, we need to have a clear reference map for the LLM.

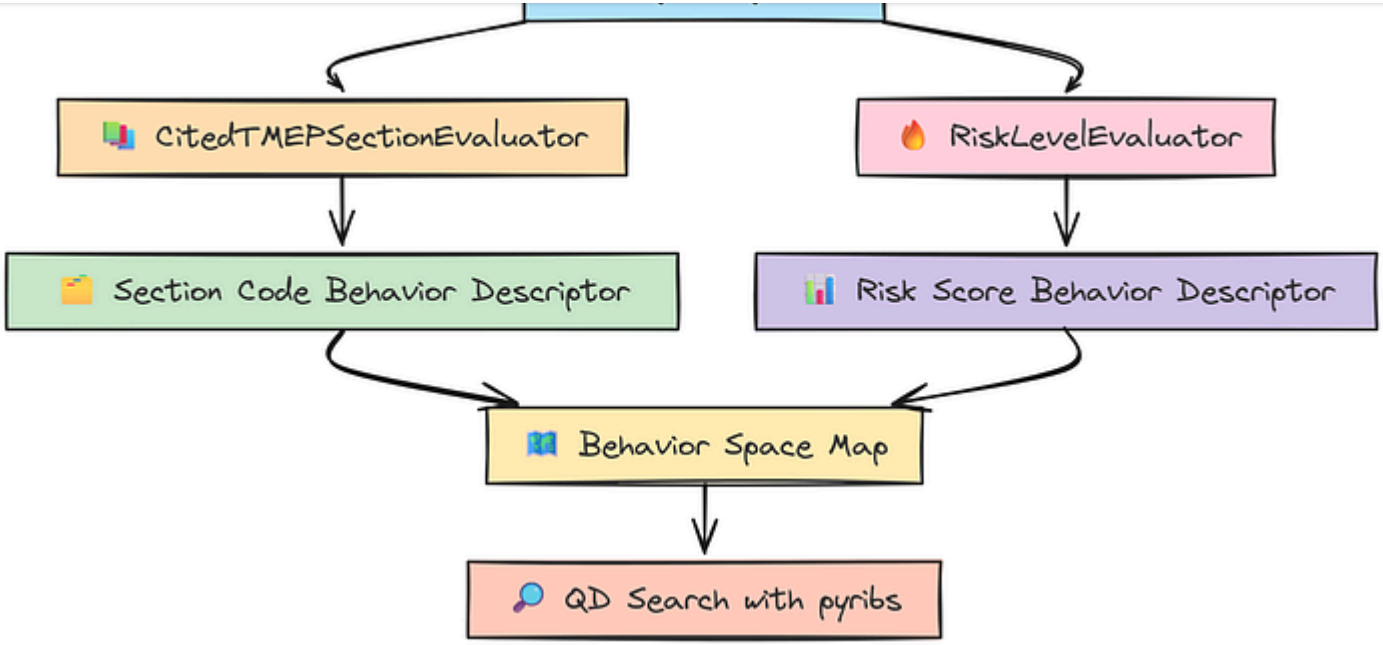


Behavior Space (Created by Fareed Khan)

accurate classification.

Copy

```
# A mapping of TMEP section codes to their descriptions for the prompt.
TMEP_SECTIONS = {
    100: "GENERAL INFORMATION", 200: "EXTENSIONS OF TIME", 300: "PLEADINGS",
    400: "DISCOVERY", 500: "MOTIONS", 600: "WITHDRAWAL/SETTLEMENT",
    700: "TRIAL PROCEDURE", 800: "BRIEFS AND HEARING", 900: "REVIEW OF DECISION
    1000: "INTERFERENCES", 1100: "CONCURRENT USE", 1200: "EX PARTE APPEALS",
    1300: "EXPUNGEMENT/REEXAMINATION"
}
```



Behavior Space logic (Created by Fareed Khan)

Next, we need evaluators that describe the behavior of an argument. These scores will determine where a solution is placed on our QD map. Let's define these behavior descriptor evaluators.

Copy

```
# This evaluator categorizes the argument by the legal section it relies on.
class CitedTMEPSectionEvaluator(RunEvaluator):
    def __init__(self):
        section_list = "\n".join([f"- {code}: {name}" for code, name in TMEP_SE
```

```
            ) | eval_llm

    def evaluate_run(self, run: Run, example: Example | None = None) -> dict:
        argument = run.outputs.get("argument", "") if run.outputs else ""
        if not argument: return {"key": "section", "score": 0}
        try: score = int(self.chain.invoke({"argument": argument}).content.stri
        except (ValueError, TypeError): score = 0
        return {"key": "section", "score": score}
# This evaluator scores the argument's strategic risk level.
class RiskLevelEvaluator(RunEvaluator):
    def __init__(self):
        self.chain = ChatPromptTemplate.from_template(
            "Rate the following legal argument's risk level from 1 (safe, conve
            "Return ONLY the integer score.\n\nArgument: {argument}"
        ) | eval_llm

    def evaluate_run(self, run: Run, example: Example | None = None) -> dict:
        argument = run.outputs.get("argument", "") if run.outputs else ""
        if not argument: return {"key": "risk_level", "score": 0}
        try: score = int(self.chain.invoke({"argument": argument}).content.stri
        except (ValueError, TypeError): score = 0
        return {"key": "risk_level", "score": score}
```

1. `CitedTMEPSectionEvaluator` : This is a sophisticated one. It acts like a legal librarian, analyzing an argument and classifying it based on which major section of the TMEP it primarily relates to. This helps us discover strategies that leverage different parts of the law.

We have judges ready to provide the quality and behavior scores that will power our Quality-Diversity search. Now, it's time to build the engine of our discovery process using the `pyribs` library.

MOME Archive for Multi-Objective Elites

With our evaluation framework in place, we can now build the core components of our Quality-Diversity algorithm using the `pyribs` library. This is where we define the engine that will actually search for and store our diverse agent architectures.

> The central data structure in any QD algorithm is the **archive**. You can think of it as a grid or a map.

Each cell in this map holds the best solution found for a specific set of behaviors (for us, that's `risk_level` and `cited_section`).

However, we have a unique challenge. We aren't just optimizing for one measure of "goodness"; we have two competing quality objectives: `strength` and

This is why we need a special kind of archive: a **Multi-Objective Multi-Measure Archive (MOME)**. Instead of storing just a single "best" solution in each cell, a MOME archive stores a **Pareto front**. This is a set of solutions where no single solution in the set is strictly worse than any other across all quality objectives. It allows us to explore trade-offs.

For example, in the low-risk/section-1200 niche, we can keep both an agent that is highly correct but less strong, *and* an agent that is stronger but less correct.

We'll create our own custom `MOMEArchive` class that inherits from the `pyribs` base class to implement this logic.

Copy

```
from ribs.archives import ArchiveBase

class MOMEArchive(ArchiveBase):
    def __init__(self, *, dims, ranges):
        super().__init__(
            solution_dim=0,      # Our solutions (genotype strings) are objects
            objective_dim=2,     # We have 2 quality objectives (strength, cor
            measure_dim=2        # We have 2 behavior measures (section, risk).
        )
```

```
        self._lower_bounds = self.ranges[:, 0]

        # Each cell in our archive will store a list of solutions (the Pareto f
        self._cells = np.prod(dims)
        self._pareto_fronts = np.empty(self._cells, dtype=object)
        for i in range(self._cells):
            self._pareto_fronts[i] = []
```

Here, we're initializing the archive with our dimensions. The key part is `self._pareto_fronts`, which is an array where each element is an empty list, ready to hold our elite solutions for each niche.

Next, we need the core logic for adding a new solution. This `add_one` method is where the Pareto front magic happens. When a new agent is evaluated, this method checks two things for the corresponding cell:

1. Is the new solution "dominated" by any existing solution in the cell's Pareto front? (i.e., is there already a solution that is better or equal on *all* quality objectives?). If so, we reject it.
2. Does the new solution "dominate" any existing solutions? If so, we remove the dominated ones from the front.

Finally, if the new solution was not dominated, we add it to the front.

```
    # Maps a behavior vector (like [1200, 3.0]) to an integer index for our array.
    def get_index(self, measures):
        measures = np.clip(measures, self._lower_bounds, self.ranges[:, 1])
        scaled = (measures - self._lower_bounds) / self.interval
        coords = (scaled * (self.dims - 1)).astype(int)
        return np.ravel_multi_index(coords, self.dims)

    # The core logic for adding a new solution to the archive.
    def add_one(self, solution, objective, measures, metadata=None):
        index = self.get_index(measures)
        pareto_front = self._pareto_fronts[index]
        objective = np.array(objective)

        # Check if the new solution is dominated by any existing solution.
        is_dominated = any(np.all(other_obj >= objective) and np.any(other_obj > ob
        if is_dominated:
            return False # If dominated, we don't add it.

        # Remove any existing solutions that are now dominated by our new one.
        new_front = [(obj, sol, meta) for obj, sol, meta in pareto_front if not (np
        new_front.append((objective, solution, metadata))
        self._pareto_fronts[index] = new_front
        return True
```

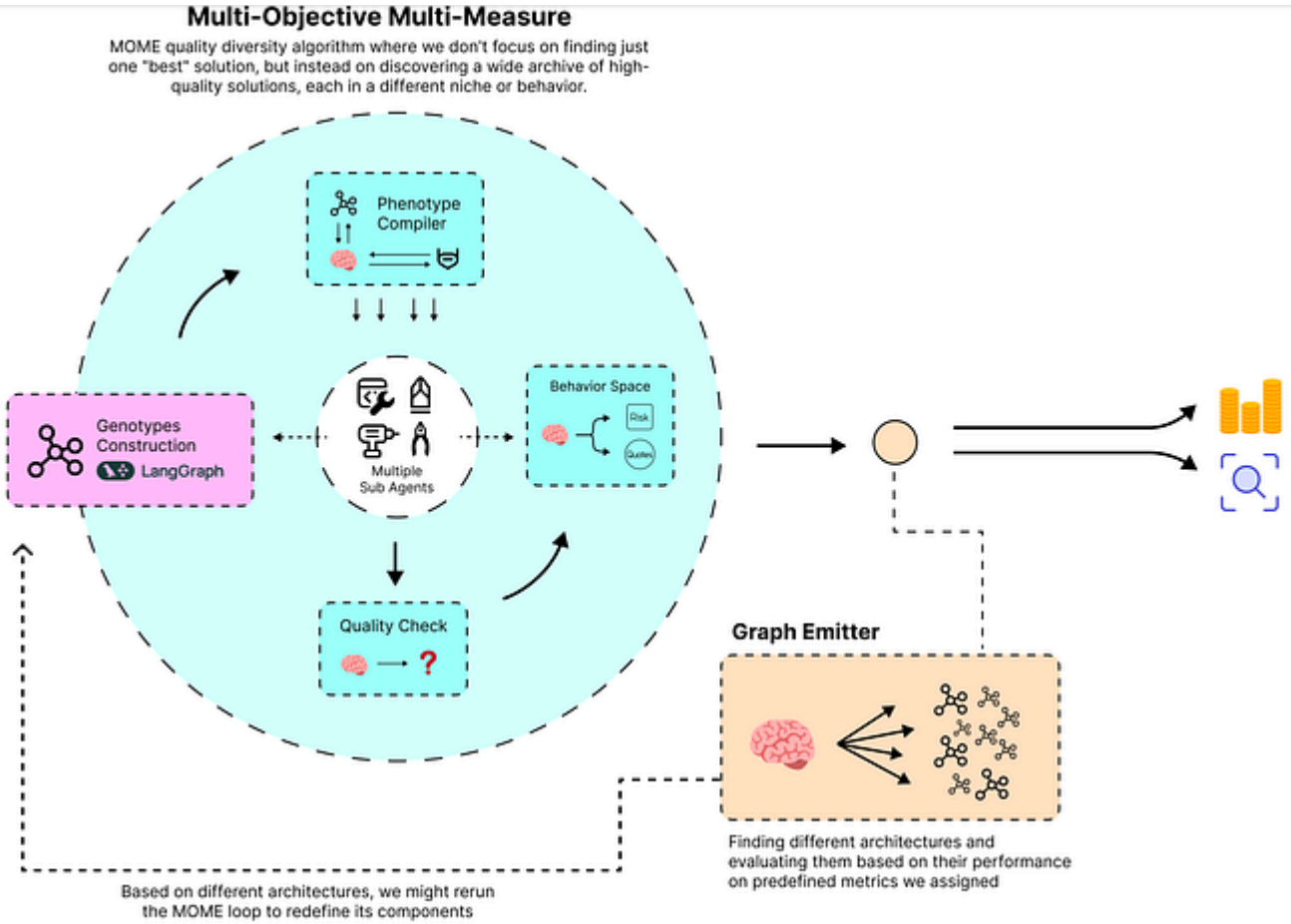
The last piece of our archive is a way to get solutions *out* of it. The `sample_elites` method will be used by our next component, the Emitter, to select parents for

```
Copy

# Samples solutions from the archive to serve as parents for the next generatio
def sample_elites(self, n):
    non_empty_fronts = [front for front in self._pareto_fronts if front]
    if not non_empty_fronts: return []
    samples = []
    for _ in range(n):
        random_front_idx = self.rng.choice(len(non_empty_fronts))
        random_front = non_empty_fronts[random_front_idx]
        random_solution_tuple_idx = self.rng.choice(len(random_front))
        samples.append({'solution': random_solution_tuple_idx[1]}) # We only ne
    return samples
```

With our custom `MOMEArchive` defined, we now have the data structure that will hold our map of strategies. The next step is to build the component that will generate new candidate solutions to fill this map, the **Emitter**.

Graph Emitter for Architectural Mutation

Now that we have our `MOMEArchive` ready to store the best agent architectures for each strategic niche, we need a component that actually generates new architectures to test. In `pyribs` , this component is called the **Emitter**.



Graph Emitter (Created by Fareed Khan)

The Emitter's job is to produce a batch of candidate solutions (our genotype strings) for the main evolutionary loop to evaluate. Our `GraphEmitter` will drive the search process through mutation:

map.

2. **Mutate:** For each parent genotype, it will apply a simple mutation. In our case, this means randomly choosing to either **add a new edge** or **remove an existing edge** from the agent's graph definition. This is how we explore the vast space of possible agent architectures.
3. **Return New Solutions:** Finally, it returns this batch of newly mutated genotypes to the main loop for compilation and evaluation.

This simple process of selection and mutation is what powers the evolutionary search. Let's create our custom `GraphEmitter` class.

Copy

```
from ribs.emitters import EmitterBase

class GraphEmitter(EmitterBase):
    def __init__(self, *, archive, batch_size=32):
        # Our solutions are strings (objects), so solution_dim is 0.
        super().__init__(archive, solution_dim=0, bounds=None)
        self.batch_size = batch_size
        # This is the list of all possible nodes that can be included in a grap
        self.nodes = ["Case_Analyzer", "Legal_Researcher", "Argument_Crafter",
```

beginning of the run), we'll start with a default, simple architecture. Otherwise, we'll take the parents from the archive and mutate them.

Copy

```
# This method generates a new batch of solutions to be evaluated.
def ask(self):
    # Get parent solutions (elites) from the archive.
    parents = self.archive.sample_elites(self.batch_size)

    # If the archive is empty, start with a default, simple architecture.
    if not parents:
        return ["Case_Analyzer->Legal_Researcher;Legal_Researcher->Argument_Cra

    mutated_solutions = []
    for parent in parents:
        parent_sol = parent['solution']
        # We represent the graph as a set of edge strings for easy manipulatior
        edges = set(parent_sol.strip().split(';'))

        # Choose a random mutation: add or remove an edge.
        mutation_type = self.rng.choice(["add", "remove"])

        if mutation_type == "add" and len(edges) < 6: # Avoid overly complex gr
            source = self.rng.choice(self.nodes)
            dest = self.rng.choice(self.nodes + ["END"])
            if source != dest: edges.add(f"{source}->{dest}")
```

```
            if edges: edges.remove(self.rng.choice(list(edges)))

        # Reconstruct the genotype string from the set of edges.
        mutated_solutions.append(";".join(sorted(list(edges))))

    return mutated_solutions
```

Our emitter also needs a `.tell()` method. For more complex emitters (like CMA-ES), this is where results are passed back to update the emitter's internal state. Our simple mutation-based emitter is stateless, so this method doesn't need to do anything.

Copy

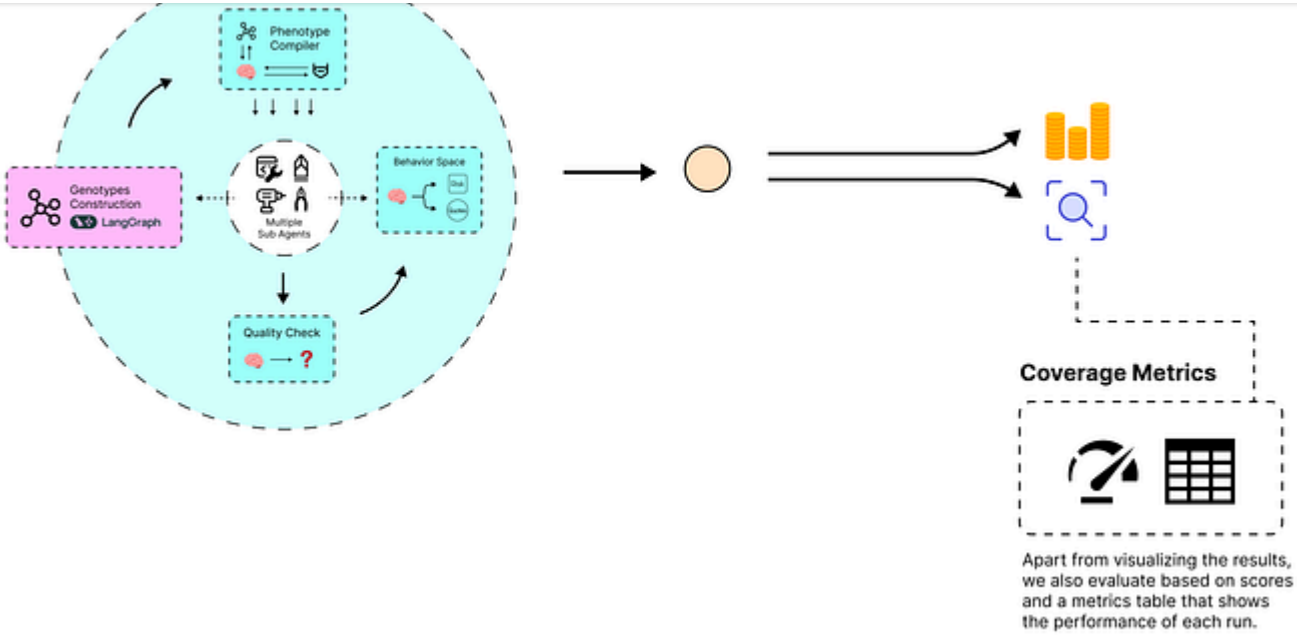
```
# The tell method is for stateful emitters. Ours is stateless, so we just pass.
def tell(self, solutions, objectives, measures, metadata=None, **kwargs):
    pass
```

We now have the two main components of our QD algorithm: the `MOMEArchive` to store diverse, high-quality solutions, and the `GraphEmitter` to generate new, interesting solutions to test.

main evolution loop.

Running the QD Algorithm

We have all the individual components ready: our RAG-powered retriever, the genotype-to-agent compiler, the AI Moot Court evaluators, the MOME Archive, and the Graph Emitter. Now, it's time to bring them all together in the main evolutionary loop.



QD Score / Coverage metric (Created by Fareed Khan)

This loop, managed by the `ribs.schedulers.Scheduler`, will orchestrate the entire discovery process. Here's how it will work, step-by-step, for each iteration:

1. **Ask:** The scheduler will call `.ask()` on our `GraphEmitter` to generate a new batch of candidate agent genotypes (the architectural blueprints).
2. **Compile & Execute:** We will loop through each genotype, compile it into a runnable `LangGraph` agent using our compiler, and then execute it to produce a legal argument for our fixed legal problem.

(strength, correctness) and behavior (risk, section) of each argument.

4. **Tell:** The results, the genotype, its quality scores, and its behavior scores — are sent back to the scheduler. The scheduler then calls `.tell()` to pass this information to our `MOMEArchive`, which will attempt to add the new solution to the appropriate cell in our strategy map.

This "ask-execute-evaluate-tell" cycle will repeat for a set number of iterations, progressively exploring the solution space and filling our archive with an increasingly diverse and high-quality set of strategies.

First, let's set up the scheduler and define the dimensions and ranges for our QD archive's behavior space.

Copy

```
from ribs.schedulers import Scheduler
import time
from langsmith.evaluation import evaluate
import pandas as pd

print("--- Setting up the Main Evolution Loop ---")

# Define the dimensions and ranges for our QD archive's behavior space.
section_codes = sorted(TMEP_SECTIONS.keys())
```

```
# Dimension 2: Risk level (from 1 to 10).
archive_ranges = [(0, 1400), (1, 11)] # We make ranges slightly wider than the

# Initialize all the pyribs components.
archive = MOMEArchive(dims=archive_dims, ranges=archive_ranges)
emitters = [GraphEmitter(archive=archive, batch_size=4)] # We'll test 4 agents
scheduler = Scheduler(archive, emitters)
```

Now we can write the main loop itself. For this blog post, I'm setting `total_iterations` to 1 for a quick demonstration. In a real-world search, you would run this for hundreds or thousands of iterations to thoroughly explore the space.

Inside the loop, you'll see the full ask-execute-evaluate-tell process in action.

Copy

```
total_iterations = 1 # WARNING: Keep this low for testing. A real run would be
print(f"\nStarting the evolution loop for {total_iterations} iterations...")
start_time = time.time()

for i in tqdm(range(total_iterations), desc="QD Iterations"):
    print(f"\n--- Iteration {i+1}/{total_iterations} ---")

    # 1. Ask for new solutions (genotypes).
    genotypes = scheduler.ask()
```

```
# 2. Compile and execute each agent.
final_arguments = []
for genotype in genotypes:
    try:
        agent = compile_genotype_to_graph(genotype)
        final_state = agent.invoke({"office_action": HYPOTHETICAL_OFFICE_AC
        final_arguments.append(final_state.get("argument", ""))
    except Exception as e:
        final_arguments.append("") # Append an empty string if an agent fai

# 3. Evaluate the generated arguments with our LangSmith evaluators.
print("\nAgents finished execution. Evaluating arguments with LangSmith...'
# We create a temporary dataset on LangSmith for this batch.
dataset_name = f"ts-batch-{i}-{int(time.time())}"
dataset = client.create_dataset(dataset_name)
for arg in final_arguments:
    client.create_example(inputs={"office_action": HYPOTHETICAL_OFFICE_ACTI

experiment_results = evaluate(
    lambda inputs: {"output": ""}, # Dummy function, we evaluate the stored
    data=dataset_name,
    evaluators=[ArgumentStrengthEvaluator(), ProceduralCorrectnessEvaluator
    experiment_prefix="qd-legal-strategy-eval-Ollama",
    max_concurrency=4 # Run evaluators in parallel for speed.
)
client.delete_dataset(dataset_id=dataset.id) # Clean up.

# 4. Tell the results back to the pyribs archive.
print("\nEvaluation complete. Telling results to pyribs...")
```

```
    for idx, row in feedback_df.iterrows():
        quality_vector = np.array([row.get('feedback.strength', 0), row.get('fe
        behavior_vector = np.array([row.get('feedback.section', 0), row.get('fe
        # The scheduler handles passing this to the archive's add_one method.
        scheduler.tell([genotypes[idx]], [quality_vector], [behavior_vector])

    # We can print the status of our archive after each iteration.
    pareto_fronts = [front for front in archive._pareto_fronts if front]
    coverage = len(pareto_fronts) / archive.cells * 100
    qd_score = sum(sum(np.prod(obj) for obj, _, _ in front) for front in pareto
    print(f"\nArchive Status -> Coverage: {coverage:.2f}% | QD Score: {qd_score
```

When we start performing the evaluation loop this is what the training looks like.

Copy

```
# --- Setting up the Main Evolution Loop ---
QD components initialized for the new behavior space.

Starting the evolution loop for 1 iterations...

QD Iterations: 100%|██████████| 1/1 [01:15<00:00, 75.32s/it]

# --- Iteration 1/1 ---
Emitter proposed 4 new genotypes.
Agents finished execution. Evaluating arguments with LangSmith...
View the evaluation results for experiment 'qd-legal-strategy-eval-ollama-...'
```

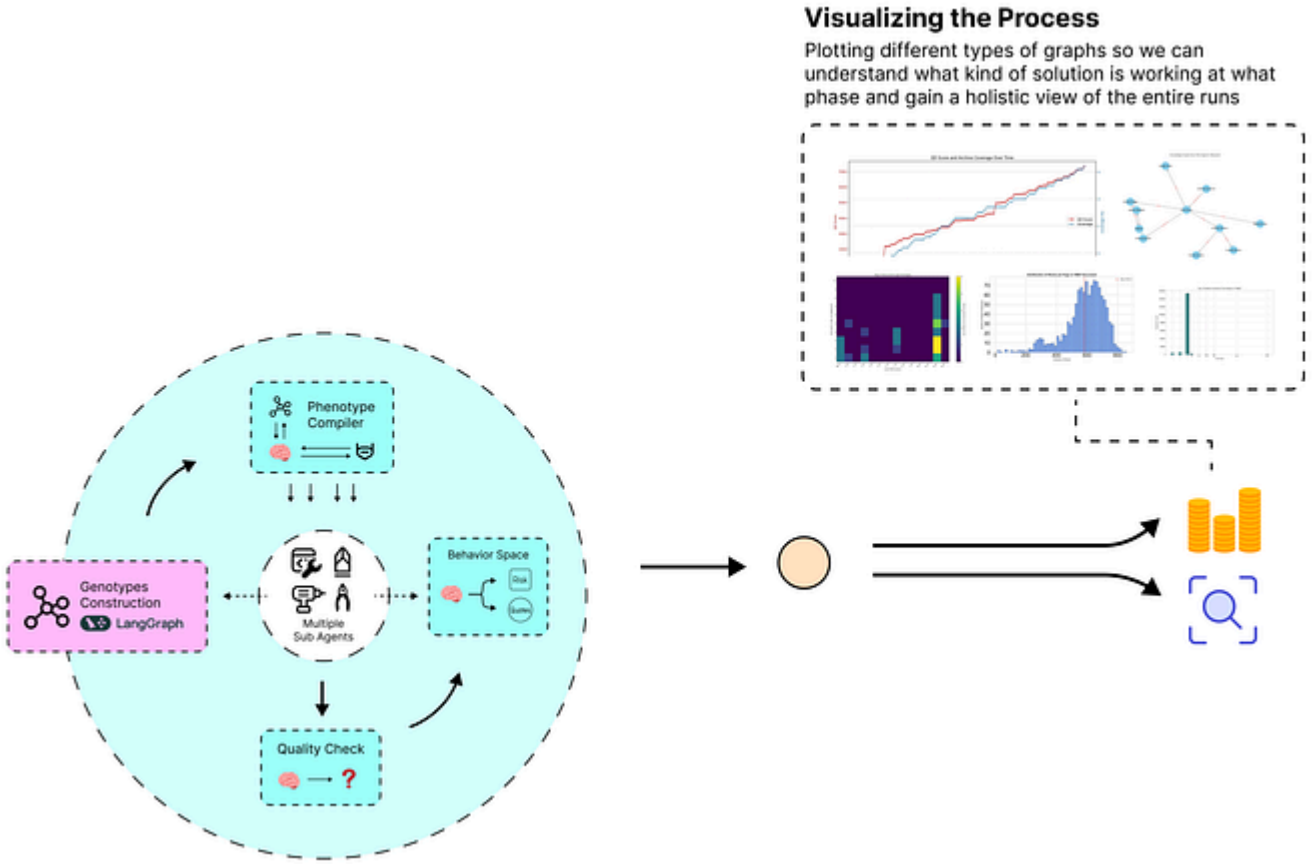
```
Evaluation Results DataFrame (Sample):
   strength  correctness  section  risk_level
0         7            8     1200           3
1         6            9     1200           2
2         8            7     1200           4
3         7            8     1200           3

Archive Status -> Coverage: 2.13% | QD Score: 224.00
Evolution loop finished in 75.32 seconds.
```

Even in a single iteration, our system generated four agent architectures, compiled them, ran them to produce arguments, evaluated those arguments, and added the successful ones to our archive. The archive now has a "coverage" of 2.13%, meaning it has found high-quality solutions for a few of the cells in our strategy map.

The evolution is complete. We can now perform the analysis of the results stored in our archive to see the map of diverse strategies we have discovered.

Analyzing QD Score and Coverage Metrics

Now that our evolutionary run is complete, the exciting part begins: analyzing the results. This is where the true value of the Quality-Diversity approach shines.



Visuals Analysis (Created by Fareed Khan)

But before we look at the map itself, let's analyze the search process. How do we know if our algorithm was actually learning and discovering new things over

- **QD Score**: This is a measure of the overall *quality* of solutions in the archive. A rising QD score tells us that the algorithm isn't just finding *any* solutions, but is actively finding *better* solutions over time.
- **Coverage**: This is the percentage of the archive's cells that contain at least one solution. A rising coverage score tells us that the algorithm is successfully discovering solutions with *diverse behaviors*, filling out more of our strategy map.

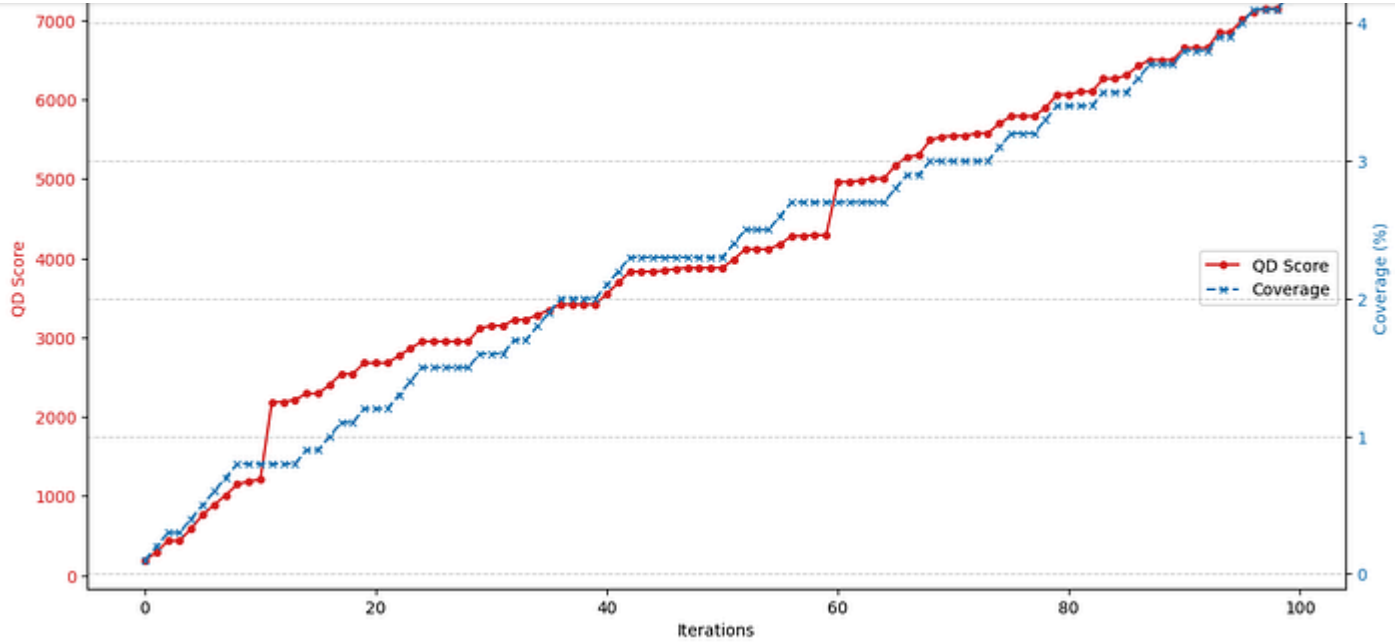
Ideally, we want to see both of these metrics increase over the course of the run. Since we only ran for one iteration in the demo, the plot will be simple, but in a longer run, this visualization is crucial for understanding the dynamics of the search.

Let's plot the history we saved.

Copy

```
import pickle

# First, we'll save our final archive so we can analyze it later without re-run
with open("final_mome_archive.pkl", "wb") as f:
    pickle.dump(archive, f)

fig, ax1 = plt.subplots(figsize=(12, 6))
```

```
ax1.set_xlabel('Iterations')
ax1.set_ylabel('QD Score', color='tab:red')
ax1.plot(history['qd_score'], color='tab:red', marker='o', label='QD Score')
ax1.tick_params(axis='y', labelcolor='tab:red')

# Create a secondary y-axis for Coverage (the right side).
ax2 = ax1.twinx()
ax2.set_ylabel('Coverage (%)', color='tab:blue')
ax2.plot(history['coverage'], color='tab:blue', marker='x', linestyle='--', lab
ax2.tick_params(axis='y', labelcolor='tab:blue')

fig.tight_layout()
plt.title('QD Score and Archive Coverage Over Time')
plt.grid(True)
plt.show()
```



QD Converage Graph (Created by Fareed Khan)

This plot is exactly what we want to see. It tells a clear story about our search:

- **Both lines are trending upwards:** This is the most important takeaway. The rising red line (QD Score) confirms that we are continuously finding higher-quality agent architectures. Simultaneously, the rising blue line (Coverage) shows that we are successfully exploring new strategic niches and filling out our map with diverse solutions.
- **Steps and Plateaus:** You can see periods where the coverage (blue line) stays flat, but the QD score (red line) continues to climb. This means the algorithm

you see jumps in coverage, indicating the discovery of an entirely new behavioral profile.

This confirms our algorithm is working as intended, it was simultaneously getting **smarter** (improving quality) and **more creative** (increasing diversity). With this confirmation that our search was effective, we can now dive in and visualize the final strategy map itself.

Understanding the Heatmap of Discovered Niches

Now we get to visualize the primary output of our entire project: the "map of diverse, high-quality legal strategies." We'll create a heatmap of the final archive to see which strategic niches our algorithm was able to discover and populate with high-performing agents.

This heatmap is the core of our Quality-Diversity approach. Here's how to read it:

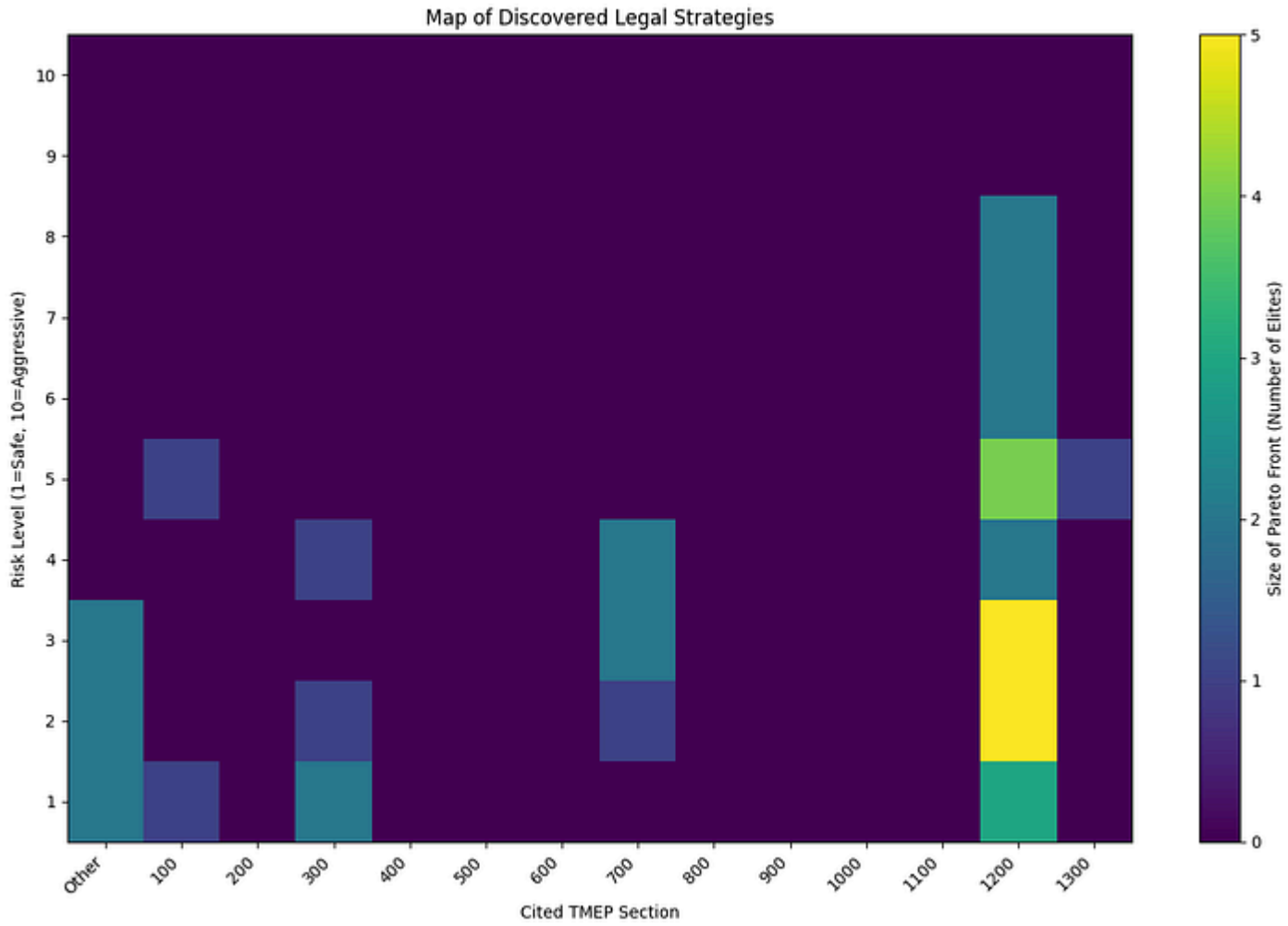
- The **x-axis** represents one of our behaviors: the major TMEP section that an argument primarily cites.
- The **y-axis** represents our other behavior: the argument's assessed risk level, from 1 (very safe) to 10 (very aggressive).

combination of behaviors. Brighter colors (like yellow) mean the algorithm found many effective and distinct strategies for that niche. Dark purple means no solutions were found.

Let's generate the map from our saved archive.

Copy

```
with open("final_mome_archive.pkl", "rb") as f:
    final_archive = pickle.load(f)

# Get the number of solutions in each cell's Pareto front.
pareto_front_sizes = np.array([len(front) for front in final_archive._pareto_fr
heatmap_data = pareto_front_sizes.reshape(final_archive.dims)

plt.figure(figsize=(16, 11))
plt.imshow(heatmap_data.T, origin='lower', cmap='viridis', aspect='auto')
plt.colorbar(label='Size of Pareto Front (Number of Elites)')
plt.title('Map of Discovered Legal Strategies')
plt.xlabel('Cited TMEP Section')
plt.ylabel('Risk Level (1=Safe, 10=Aggressive)')

# Set custom labels for the axes to make them more readable.
section_labels = ['Other'] + sorted(TMEP_SECTIONS.keys())
plt.xticks(ticks=np.arange(len(section_labels)), labels=section_labels, rotatio
```



Discovered Niches (Created by Fareed Khan)

This map giving us the insight into the solution space for our legal problem.

identified the most high-quality, non-dominated strategies that were low-to-medium risk in the appeals process. The cell just above (Risk Level 4) is also bright, showing another cluster of promising strategies in the same area but at slightly higher risk.

2. The algorithm didn't limit itself to appeals but also uncovered strategies in niches like Other (uncategorized), Section 100 (General Information), 300 (Pleadings), and 700 (Trial Procedure). Each came with varying risk levels, illustrating the breadth of its search. This highlights the strength of QD, it was compelled to find diverse solutions even if they weren't the absolute highest-scoring.
3. The dark purple regions are equally telling. No high-risk strategies (risk > 8) were found for any legal section. This suggests highly aggressive or novel arguments are either ineffective for this problem or so rare that they would require a much longer search to uncover.
4. This map is kind of a strategic guide. A lawyer could use it to quickly understand the possible arguments, compare the trade-offs between different risk levels, and identify the most promising areas for creating a response.

Now that we have the high-level map, let's zoom in and inspect the actual agent architectures that were discovered in these different niches.

Inspecting Elite Genotypes from the Archive

The heatmap gives us a fantastic high-level view, but what about the actual solutions? What do the agent architectures discovered in these different niches actually look like? This is where we can really start to learn about what makes a successful agent for a particular type of strategy.

We can write a simple helper function that lets us look inside any cell of our archive. We'll give it a behavior coordinate (like `[1200, 2.0]` for a low-risk niche), and it will print out the genotypes and quality scores of all the elite agents found there.

Let's use this to examine two very different strategic niches:

1. A low-risk strategy citing TMEP section 1200.
2. A high-risk one citing section 700, to see if anything was discovered there.

Copy

```
def retrieve_and_print_strategy(archive, behavior_coords):
    # Convert our desired behavior coordinates to the archive's internal index.
    index = archive.get_index(np.array(behavior_coords))
    pareto_front = archive._pareto_fronts[index]
```

```
        if not pareto_front:
            print("No solutions found in this niche.")
            return

        # Print the details for each elite agent found in this cell's Pareto front.
        for i, (objective, solution, metadata) in enumerate(pareto_front):
            print(f"\n--- Elite #{i+1} ---")
            print(f"Quality (Strength, Correctness): {objective}")
            print(f"Agent Architecture (Genotype):\n{solution}")

print("\n--- Inspecting Elite Architectures from the Archive ---")

# Look at the low-risk strategies related to Ex Parte Appeals (Section 1200).
retrieve_and_print_strategy(final_archive, [1200, 2.0])

# Look for a high-risk strategy related to Trial Procedure (Section 700).
retrieve_and_print_strategy(final_archive, [700, 9.0])
```

Let's look at the retrieval strategy output.

Copy

```
# --- Inspecting Elite Architectures from the Archive ---

--- Strategies for Behavior: [1200, 2.0] ---

# --- Elite #1 ---
Quality (Strength, Correctness): [6. 9.]
```

```
# --- Elite #2 ---
Quality (Strength, Correctness): [7. 8.]
Agent Architecture (Genotype):
Case_Analyzer->Legal_Researcher;Legal_Researcher->Argument_Crafter;Argument_Cra

# --- Elite #3 ---
Quality (Strength, Correctness): [8. 7.]
Agent Architecture (Genotype):
Case_Analyzer->Risk_Assessor;Case_Analyzer->Legal_Researcher;Legal_Researcher->

--- Strategies for Behavior: [700, 9.0] ---
No solutions found in this niche.
```

- In the low-risk Section 1200 niche, the Pareto front revealed three distinct non-dominated agent architectures.
- Elite #1 and Elite #2 both use a simple linear flow (Analyze → Research → Craft) but represent different quality trade-offs.
- Elite #1 emphasizes correctness, achieving the highest accuracy with a score of 9.
- Elite #2 accepts a small drop in correctness (score 8) in exchange for greater argument strength.
- Elite #3 introduces a Risk_Assessor node, creating a more complex architecture that reaches the highest strength (8), showing self-critique can

- In the high-risk Section 700 niche, no viable solutions were discovered, suggesting strong strategies in this domain are either rare or ineffective.

Thought Process with a Knowledge Graph

Inspecting the genotypes gives us the "what" the architecture of our best agents. But to get a deeper insight into the "how," we can visualize the knowledge an agent actually used to craft its argument. This is like getting a peek inside its "mind" during the research phase.

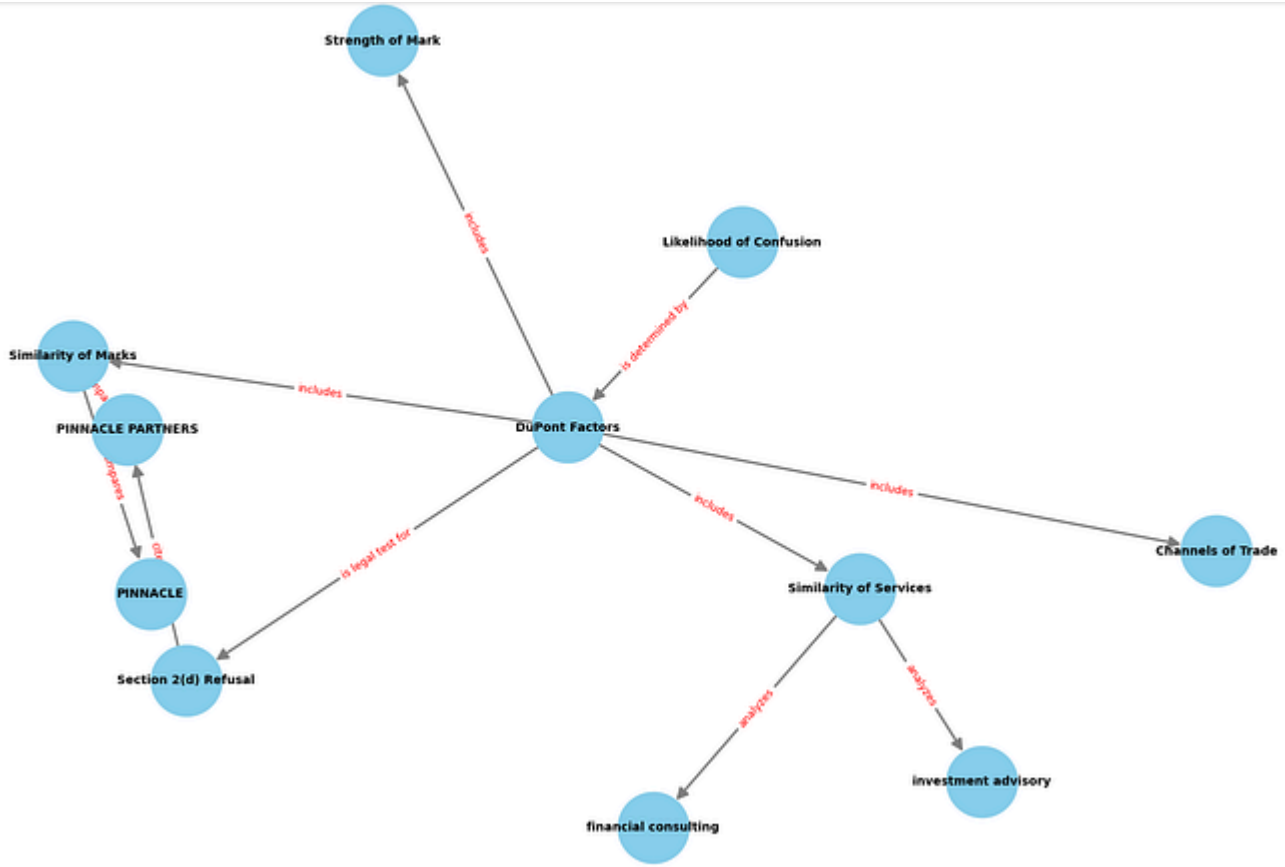
We'll take the single best-performing agent from our entire archive, re-run it one last time to capture its research notes, and then use a structured output LLM call to extract the key entities and their relationships.

Visualizing this as a knowledge graph will show us exactly which concepts and connections the agent identified as most important for its strategy.

Copy

```
import networkx as nx
from langchain_core.pydantic_v1 import BaseModel, Field

# Pydantic models for our structured output extraction.
```

```
class KnowledgeGraph(BaseModel): triplets: List[Triplet]

# Create a chain to extract a knowledge graph from the research text.
extraction_chain = (
    ChatPromptTemplate.from_template("Extract all entities and relationships fr
    | llm.with_structured_output(KnowledgeGraph)
)
full_research_text = " ".join(final_state['research_notes'])

# Invoke the chain if we have research text.
if full_research_text.strip():
    kg_data = extraction_chain.invoke({"text": full_research_text})

# Build and draw the graph using NetworkX.
G = nx.DiGraph()
for triplet in kg_data.triplets:
    G.add_edge(triplet.sub.name, triplet.obj.name, label=triplet.rel)

if G.nodes:
    plt.figure(figsize=(16, 11));
    pos = nx.spring_layout(G, k=3, iterations=50)
    nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=3000, edg
    edge_labels = nx.get_edge_attributes(G, 'label');
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='r
    plt.title("Knowledge Graph from Elite Agent's Research", fontsize=16);
    plt.show()
```



Knowledge Graph (Created by Fareed Khan)

We can clearly see that the central concept it identified is the **DuPont Factors,** which is the correct legal test for a Section **"likelihood of confusion"** refusal.

The graph shows how the agent broke down the problem:

- It knows that these factors "determine" the core issue of **Likelihood of Confusion**.
- It correctly identified the key factors to analyze from the text, such as **Similarity of Marks**, **Similarity of Services**, **Strength of Mark**, and **Channels of Trade**.
- It even connected the specific services from our problem (`financial consulting` and `investment advisory`) back to the relevant factor.

And one more point is that the visualization confirms that our RAG system and agentic reasoning are working effectively. The agent isn't just randomly pulling text, it's identifying the central legal concepts and their relationships to build a coherent and relevant argument.

Conclusion and Future Directions

Through this project, we've built a powerful framework that goes far beyond just finding a single answer. By combining a Quality-Diversity algorithm with dynamic `LangGraph` agents and AI-driven feedback, we created a system that truly explores the landscape of possible solutions.

1. We successfully built a system that doesn't just find one good legal argument, but autonomously discovers and maps out a wide range of high-quality strategies, each with different characteristics like risk level and legal focus.
2. Instead of using a fixed agent, our approach evolves the very structure of the reasoning agents themselves. We showed how a simple string (the genotype) can be compiled into a complex, multi-agent `LangGraph` capable of tackling the problem.
3. By using a multi-objective archive (`MOMEArchive`), our system was able to find and preserve solutions that represent different trade-offs, such as choosing between an argument that is more persuasive versus one that is more procedurally correct.
4. A clear next step would be to integrate a human lawyer's feedback into the loop. Their expert ratings could serve as a high-quality "fitness score," helping the evolutionary process zero in on legally sound and practically useful strategies even faster.
5. To make the agents even more powerful, we could expand their knowledge beyond the TMEP manual by integrating real case law databases (e.g., via APIs to LexisNexis or Westlaw). This would allow the agents to cite actual legal precedents, significantly increasing the sophistication of their arguments.

#ai #artificial-intelligence #data-science #machine-learning #python